Software Engineering for Augmented Reality – A Research Agenda

INGO BÖRSTING, MARKUS HEIKAMP, MARC HESENIUS, WILHELM KOOP, and VOLKER GRUHN, University of Duisburg-Essen, Germany

Augmented reality changes the way we perceive reality and how we interact with computers. However, we argue that to create augmented reality solutions, we need to rethink the way we develop software. In this paper, we review the state of the art in software engineering for augmented reality applications, derive open questions, and define a research agenda. For this purpose, we consider different engineering phases and evaluate conventional techniques regarding their applicability for AR development. In requirements engineering, we found the integration of AR experts and the associated collaboration between actors to be of key aspect in the development process. Additionally, requirements about the physical world must be considered, which in turn has a huge impact on UI design. The relevance of the physical environment is not yet sufficiently addressed in applicable techniques, which also applies to current implementation frameworks and tools, complicating the AR development process. When evaluating AR software iterations, we found interaction testing and test automation to have great potential, although they have not yet been sufficiently researched. Our paper contributes to AR research by revealing current core challenges within the AR development process and formulating explicit research questions that should be considered by future research.

CCS Concepts: • **Computing methodologies** → **Mixed / augmented reality**; • **Software and its engineering** → *Software development techniques*; *Software creation and management*.

Additional Key Words and Phrases: software engineering, augmented reality, methods, frameworks, tools

ACM Reference Format:
Ingo Börsting, Markus Heikamp, Marc Hesenius, Wilhelm Koop, and, Volker Gruhn. 2022. Software Engineering for Augmented Reality – A Research Agenda. *Proc. ACM Hum.-Comput. Interact.* 6, EICS, Article 155 (June 2022), 34 pages. https://doi.org/10.1145/3532205

1 INTRODUCTION

Augmented Reality (AR) enhances users' physical environment with virtual content and is used in a wide area of applications like cultural heritage, medicine, or industry. AR technology assists surgeons by simulating operation scenarios [38], supports quality assurance in production planning [44], or aids in transmitting educational content [19]. Furthermore, AR is a constant topic in more futuristic use cases, e.g. to navigate pedestrians in autonomous traffic [53].

Nevertheless, the development of AR software remains a challenge. AR-specific characteristics like novel interaction techniques, registration of the physical world, or device diversity result in advanced requirements for software development, especially compared to conventional information systems [30]. Furthermore, interaction modalities such as speech, gestures, touch, or gaze require

Authors' address: Ingo Börsting, ingo.boersting@uni-due.de; Markus Heikamp, markus.heikamp@uni-due.de; Marc Hesenius, marc.hesenius@uni-due.de; Wilhelm Koop, wilhelm.koop@uni-due.de;
Volker Gruhn, volker.gruhn@uni-due.de, University of Duisburg-Essen, Schützenbahn 70, 45127 Essen, Germany.

2573-0142/2022/6-ART155 $15.00
https://doi.org/10.1145/3532205

complex processing procedures, intensive testing, and must be tailored to different user groups and tasks. Some AR applications even enrich interaction concepts to the extent that everyday objects can be used as an input modality [115]. Since AR is a digital enhancement of the physical world, the users' real environment is an essential part of AR systems. This leads to significant challenges in the software development process, since the physical environment has to be present in various phases [68]. Also, challenges in usability engineering arise, since concepts for interacting with physical and virtual objects need to be established. Some of these challenges have already been approached, like attempts to derive usability guidelines from user-based studies [99], but are mostly limited to a few User Interface (UI) elements, certain devices, or application domains. Thus, AR creators face a lack of concrete design guidelines [6]. As a result, recent AR software suffers from weaknesses in usability and functionality [90].

Currently, it remains unclear whether existing software development methods and tools are sufficient for the development of AR applications, especially considering aforementioned AR-specific characteristics. Thus, in this paper we review software engineering tools and methods regarding their applicability for the development of AR software. Here, we focus on established methods in individual development phases and not on the process framework itself. In general, we assume that existing software engineering techniques need to be adapted to the special AR characteristics to enable an efficient development process.

The development of AR applications is widely considered as an iterative process, since user needs must be continuously evaluated when working with emerging and immersive technologies [5, 39, 63]. Even though strict development phases have been replaced by flexible iterations in agile software development, every agile development iteration involves established activities, albeit in a shortened form. The separation of development phases in this paper serves solely to structure our findings. Here, we see requirements elicitation as a phase that iterations usually start with. Since AR applications comprise a wide range of interaction possibilities, we consider User Interface (UI) design as another crucial development phase, followed by implementation and testing. We perceived deployment to be strongly intertwined with implementation within our projects and thus consider deployment activities within the implementation section. Therefore, we will focus on the engineering phases *Requirements Engineering*, *UI design*, *Implementation*, and *Testing* in detail. For each phase, we discuss current findings from the scientific software engineering community and highlight the consideration of AR-specific characteristics within these approaches. We selected scientific literature based on several criteria, including the topicality, scientific validity, and reputation of the source. In order to capture the current state of research, the thematic relevance was a particularly crucial criteria. In addition to reviewing current research results, we developed two individual AR applications to explore the development process and gather hands-on experience (see Section 2).

When discussing specific software engineering activities, it is conceivable that some activities can be assigned to several phases (e.g. prototyping). We assigned these activities to the phases in which we suspect the greatest value for the AR development.

The main objective of our research is the formulation of unsolved research questions that remain in AR research. For this purpose, we provide an overview of open research questions at the end of each chapter. We thus contribute to the field of software engineering for AR systems by:

(1) reviewing the current research landscape concerning AR software development, structured by development phases and techniques.
(2) discussing own experiences in AR development across device groups and reporting identified challenges.

(3) defining open research questions and a comprehensive research agenda that reflects current AR research gaps.

In the following sections, we will discuss the current AR landscape in general and different development phases in detail. We first present our investigated AR projects in Section 2 and review related research regarding software engineering concepts for AR in Section 3. We then examine the application of conventional requirements engineering techniques for AR systems in Section 4. In Section 5, we discuss design activities that we assume to be crucial for the development of AR UIs. We then investigate implementation activities in Section 6 and discuss testing as an AR development phase in Section 7. We conclude with an outlook on future work in Section 8.

2 AUGMENTED REALITY PROJECTS

Within our research, we developed two different AR applications to investigate the applicability of classical software development methods in the context of AR. Each of our projects focused on different core aspects to generate a diverse picture about the current AR development process. Both projects were implemented over a six-month period by groups of university students which were monitored by four supervisors who acted as stakeholders and ensured the accurate execution of engineering methods. Within these projects, the supervisors used observation techniques to capture relevant learnings.

The core objective of our first AR project was to investigate development methods involving novel AR hardware. For this purpose, a Head-Mounted Display (HMD) was predefined as the target device (see Table 1) to explore innovative interaction forms and tracking methods. Within this project, the fusion of the real and virtual world was focussed in particular, involving special virtual objects, which were supposed to not only represent basic information, but also to act autonomously. To address these core requirements, a virtual AR pet was developed, which appears within the identified real world and either interacts with physical artifacts autonomously, or responds to user input. Users might interact with the pet via different forms of interaction, e.g. to select virtual food, which can be fed to the pet.

Within our second AR project, the predefinition of a target device was intentionally omitted in order to integrate the hardware decision into the development process. The core objective of this project was the exchange of information about both worlds (i.e. physical and virtual) between multiple AR users in real time (see Table 1). Thus, the desired AR application was supposed to enable users to interact with shared virtual objects within the same physical environment, requiring a shared coordinate system between devices, which presupposes a mutual perception of the physical world. In order to investigate these requirements, we decided to bring the popular game *Pong* to AR. Within our adaptation, a designated physical surface serves as the playing field, which is automatically outlined with virtual edges after the manual scanning process. Information about the perimeter of the playing field is then synchronized with another device, allowing both players to compete on a shared augmented playground within the physical environment. Besides sharing this initial information, the real-time synchronization of the virtual ball between devices was at the core of our development work.

In both AR projects, all development phases were run through in accordance with the described preconditions. The execution of classical development methods was observed and documented. Hands-on experiences from our projects are highlighted in the following chapters and embedded in the context of related literature.

	Virtual Pet	AR-Pong
Description	Interaction with a virtual pet moving in the user's physical environment	Adaption of the classic game Pong to a multi-user mobile AR application
Device	HMD	Smartphones
AR-specific characteristics	• Multimodal interaction with virtual objects • Interaction of virtual objects with the physical world	• Multi-user interaction with shared AR content • Real-time synchronization of virtual objects between devices • Real-time synchronization of real world information

Table 1. Overview of the prototypical AR applications

3 RELATED WORK

In this paper, we highlight the current state of research and challenges in AR development across various engineering phases. In general, research that examines the overall AR engineering process is still underrepresented. Nevertheless, few works recently appeared discussing the present state of AR. As an example, Ashtari et al. [6] conducted an extensive study to identify challenges in AR development through interviews with actors from different engineering phases, resulting in eight main barriers to AR development. These barriers include the lack of design guidelines, the complex consideration of the physical world, and the challenging testing process. Ashtari et al. especially focus on methods and tools, particularly in design, implementation, and testing. Furthermore, Krauß et al. [75] conducted interviews with different process actors and elicited key challenges, especially regarding actors' interdisciplinary collaboration. Here, the authors identify the misconception of the AR medium, the lack of supporting tools, and a missing shared language between actors as core challenges. We contribute to the above-mentioned findings by exploring established activities in each software development phase and examining their suitability for application to AR projects through hands-on experience combined with a broad review of related work, resulting in the revelation of specific research demands per development phase.

When considering the development process of novel technologies like AR, an examination of related technologies, such as Virtual Reality (VR), is inevitable. There are similarities in software development, e.g. novel interaction techniques or 3D content modeling, which might allow the transfer of research findings from one domain to another. Within the VR research field, Jerald [63] describes an iterative *Define-Make-Learn-Cycle* including requirements engineering and prototyping as well as the acquisition and application of user-based data. Farinazzo Martins et al. [34] adapted an iterative engineering process to the special characteristics of the VR technology, including requirements analysis, design, implementation, and testing. Mentioned approaches are either high-level process frameworks or process guidelines tailored to the specific VR technology, AR-specific characteristics are thus not considered. It has to be verified whether these proposed approaches can be transferred to the software development of AR applications.

Nevertheless, most current AR and VR research focuses rather on specific challenges in individual development activities than on the development process as a whole. Before we discuss these

activities in the subsequent chapters, core aspects of individual AR development phases and representative research will be presented in the following.

In requirements engineering, related research deals with challenges that arise from the fact that AR remains a novel technology, including interaction techniques and lack of usability standards [30]. Therefore, defining suitable usability concepts is an essential aspect of requirements engineering. Similar topics were covered within the VR domain, resulting in rapid prototyping tools to simplify requirements engineering [69]. The inclusion of the physical world in requirements engineering activities is another central research topic. Farinazzo Martins et al. [34] describe the engineering of environmental requirements to define the application context. Neira-Tovar and Castilla Rodriguez [89] add the concept of *Equipment Requirements*, defining environmental and hardware-related requirements, which might also be of interest for the requirement engineering of AR systems.

Furthermore, current AR research lacks approaches that consider the design process in its entirety. As one of the few examples, Díaz and Aedo [31] apply software engineering practices to define a holistic process for designing AR systems, facilitated by the proposed tool *CoDICE*, which supports virtual co-design assisted by shared documentation. In related literature, the lack of usability standards was identified as a key challenge. Here, a few design guidelines have been formulated to simplify the engineering process in AR development. Piumsomboon et al. [93] made a first step on standardizing usability concepts by eliciting over 40 gestures preferred by users for AR applications. Billinghurst et al. [11] describe general design guidelines, including physical and virtual objects as well as interaction metaphors. Here, the interaction metaphor defines the interaction with virtual objects in a physical environment. Ko et al. [73] formulate a set of user-centric usability principles for the development of mobile AR applications. These categories include the presentation of information, cognition, support components, user interaction, and usage aspects. As one of the first commercial players, Adobe published a general guide to AR UI design [2], emphasizing, e.g., application onboarding, intuitive interactions, and immersive AR experiences. However, the scarce research in this area has not yet led to the dissemination of usability standards. Only high-level approximations to detailed standards can be derived.

The implementation of AR software is a central research topic, mainly focussed on technology driven aspects. These include hardware- or software-issues, as stated by Dünser et al. [30], especially regarding registration and tracking methods [35, 42, 67]. Nevertheless, some approaches have been proposed that address the complexity of AR software implementation, e.g., the framework *AMIRE* allows for authoring AR content without programming skills [3], and *DART* supports the rapid implementation of design ideas using a building blocks approach [83]. However, these approaches are rarely used in current AR development, since most toolkits are specialized in creating prototypes and are not suitable for developing deployable software [40]. In addition, these frameworks usually address individual core aspects of software development, such as collaboration or 3D modeling. Comprehensive approaches are underrepresented.

When it comes to software testing, current AR research concentrates on usability studies based on end user feedback. Unit testing of AR applications and other code testing techniques are underrepresented. As AR applications involve physical environments, in-situ user-based studies in AR are laborious and time-consuming [39, 68, 83]. Approaches to prevent frequent field visits need to be considered within process frameworks, emphasizing the need for suitable test automation technology.

As stated, few works address the AR development process in its entirety. We contribute to current AR research by considering the AR development process and investigating the applicability of established engineering techniques. For this purpose, we combine hands-on experience from different AR projects with a broad review of current research. This results in a research agenda for each development phase with explicit research questions to be considered in future work.

In the following, different development phases are considered in the order in which they are usually performed in a development cycle. The following chapter thus begins with a review of requirements engineering techniques and their application within AR projects.

4 REQUIREMENTS ENGINEERING

Software engineering projects usually start with a phase in which participants identify, review, discuss, understand, and document system requirements. Systematic techniques are used to proactively identify and document end-user needs [1]. Here, activities that aim at resolving inconsistencies as well as verifying, validating, and documenting requirements are an essential part [33].

In this section, we highlight common requirements engineering methods regarding their suitability for AR software development. We will first consider brainstorming as a creative technique in Section 4.1 and then discuss the application of structured workshops within AR projects in Section 4.2. Furthermore, interviews as a technique to gather detailed requirements are highlighted in Section 4.3. The adequacy of these techniques for AR software development is examined, special characteristics observed, and arising questions discussed in Section 4.4, resulting in a list of open research questions still to be investigated in requirements engineering for AR.

4.1 Brainstorming

Brainstorming is a creativity technique usually performed in early stages of the software development process to gain a first vision of the software. Here, a small group of participants generates as much ideas as possible within a short time frame. Ideas and opinions are formulated on the basis of a start statement, followed by a subsequent clustering and evaluation of expressed ideas [111]. According to Osborn [91], a brainstorming session should encourage the modification and merging of ideas, focus on quantity as well as desire unusual ideas whereas the criticism of ideas should be prevented.

We assume brainstorming to be well suited to generate requirements for AR software, since AR as an novel technology offers room for creativity and has a high potential for uncovering excitement attributes (see [22]) that usually become explicit in brainstorming sessions. Since limits of novel technologies like AR are usually not assessable by users, we assume the brainstorming technique to encourage a restriction-free formulation of creative ideas.

We conducted brainstorming sessions within the first development iteration of all our mentioned AR projects. In this stage, only the overall objective of each AR project was set, resulting in a main requirement that served as a starting point for the brainstorming session. Further requirements needed to be engineered in order to gain a comprehensive set of software requirements. Within our AR projects, brainstorming was conducted in small groups consisting of stakeholders and software engineers. Given the main project requirement as a starting point, all participants were asked to articulate ideas and opinions regarding additional requirements. The formulated requirements were first collected and documented on a pinboard, where duplicates were eliminated. Requirements were then clustered by defining epics. As an example, the requirements for our multi-user AR Pong game where clustered in categories concerning *User Interface*, *Game Mechanics*, *Tracking of the Real World*, *Networking*, *Game Logic* and *Non-functional* requirements.

We found brainstorming to be a useful technique to reveal requirements. In contrast to conventional software engineering, the consideration of AR-specific requirements, like interacting with the physical world, have taken a lot of time within the brainstorming session. Since AR systems usually involve multiple modalities, requirements regarding the interaction concept were discussed extensively. These discussions were also based on the difficulty that usability standards for AR applications have not yet been established (see Section 1). In order to evaluate certain AR-specific

requirements, the acquisition of knowledge about AR characteristics was required. Thus, we observed that brainstorming was particularly helpful in later iterations, where participants already shared a basic understanding of the capabilities AR provides.

In total, we experienced brainstorming to be a useful technique in the early stage of AR software development in order to gain first design ideas and uncover requirements. As assumed, brainstorming enabled participants to generate excitement attributes, as the novel AR technology allows for a wide creative freedom. Here, brainstorming was mainly characterized by AR-specific elements, such as interaction concepts or environmental requirements. However, the brainstorming process revealed that advanced know-how about how AR works and what possibilities it offers is needed when gathering requirements for AR-applications. This either requires the involvement of specialists or the generation of knowledge across iterations in order to evaluate suitable requirements.

4.2 Structured Workshops

Structured workshops enable the generation of requirements within a group of stakeholders. Here, instant feedback fastens the process of finding a solution for formulated requirements. Thus, structured workshops are especially useful in early stages of the software development process, when technical or process-related aspects are discussed [111]. Lockton and Lallemand [82] showed that workshops are an appropriate instrument to develop and evaluate tools with participants. They emphasize the special importance of domain experts in structured workshops, as these participants can generate direct feedback on benefits of ideas. Several approaches for structured requirements elicitation workshops have been developed. As an example, Book et al. [14] present a workshop format called *Interaction Room*, where small interdisciplinary groups gather in a physical room and discuss requirements. Here, two coaches maintain an atmosphere in which equal communication is encouraged, so that all stakeholders are able to collaboratively formulate system requirements. Expense and risk drivers become explicit through subsequently annotating generated requirements within the workshop format. Furthermore, the *Interaction Room* encourages informal modeling.

We conducted structured workshops in all our mentioned AR projects in order to generate and evaluate requirements. Documented ideas formulated in the brainstorming sessions served as a foundation for these workshop sessions. As a first step, participants refined these rough ideas into user stories. This process was led by a workshop coach.

Next, participants discussed the merging of ideas and the formulation of joint user stories. A guided prioritization of user stories was performed according to the *MoSCoW* methodology [23]. Within this process, we found AR-related knowledge to be required in order to clarify requirements, especially in hardware-related discussions regarding potential devices and their functional scope and built-in sensors. Although technical aspects should usually not be part of requirements engineering, we found their inclusion to be essential in AR since they define the limits of possibilities, especially for advanced requirements, like the multi-user synchronization in our AR Pong application. Evaluating technical requirements regarding their feasibility was difficult, given the diversity of current AR devices. As a fact, current AR devices differ in several aspects, such as the tracking of the physical environment or supported interaction methods.

In order to refine hardware-related requirements, a subsequent technical workshop has been conducted to determine which AR devices would be most efficient for formulated requirements of our AR Pong application, since wrongful technical decisions are difficult to revert in later process stages. Within the technical workshop, participants were divided into two groups and assigned to either an HMD or a smartphone. Both groups were asked to build a simple prototype, representing a virtual sphere that approaches a wall and bounces off it. Hardware-related aspects like the tracking of the physical world, the object detection range and the logging of position data, e.g. the user's and the virtual objects' position, were examined and subsequently compared between device groups.

As a result, both groups were able to implement basic AR functionality during the workshop and investigate hardware-related aspects. However, we observed that the fixed time frame of four hours was insufficient to investigate all hardware-related aspects. This especially applies for the HMD group, as the novelty of the hardware has led to a longer training period, which has delayed the actual prototype creation. The evaluation of all relevant technical aspects would require further and longer workshop sessions, which increases the expenditure of technical requirements evaluation. This applies especially to larger AR projects with diverse technical requirements and a wider set of potential devices. Although it is likely that specific known hardware is targeted in certain (e.g., industrial) domains, it can be assumed that the constant engagement with emerging devices is necessary in the highly dynamic AR hardware landscape, regardless of the application domain. We found the diversity of AR devices to cause the need for an extensive evaluation of technical requirements in order to avoid erroneous technical decision in early project stages. Nevertheless, the findings from our technical workshop were sufficient for the evaluation of basic requirements within our rather small project.

4.3 Interviews

An interview is a structured oral survey to either examine a topic with individuals in detail or to gather and compare opinions of different individuals independently of each other [111]. Within the software development process, interviews aid in engineering detailed requirements through discussing specific questions. Interviews are the most performed technique within requirements engineering [60], since they can be performed in every stage of the development process. Here, unconscious requirements can become conscious through targeted inquiries. Since AR remains a novel technology, we assume uncertainties and unconscious requirements to frequently occur in AR software development.

We conducted interviews at different stages of the software development process in all or mentioned AR projects in order to gain functional requirements. Several interviews including different stakeholders were conducted at the beginning of each development iteration in order to specify the iterations' detailed requirements. Recurring interviews turned out to be helpful, since every development iteration led to gained knowledge about technical characteristics which influenced requirements for following iterations. At the beginning, interviews focussed on AR-specific requirements like multimodality or environmental requirements. Since the knowledge about AR-specific characteristics has only evolved over several iterations, AR-specific expertise would have been helpful in the early stages of the development process to fasten the process of technical aspects clarification.

4.4 Findings

The techniques presented in this section reveal interesting aspects and essential questions regarding requirements engineering for AR systems. As we observed in the brainstorming and workshop sessions, the discussion of AR-specific characteristics needed a lot of time. The fact that AR is a novel technology, including new devices and novel interaction forms, led to the difficulty of assessing the feasibility of certain requirements. A similar observation is described by Neira-Tovar and Castilla Rodriguez [89], who state that software engineers need to validate hardware-related aspects in order to refine requirements, which is impeded by the diversity of AR devices. This becomes even more essential when multiple devices are to be connected, e.g. to allow bidirectional interactions [70].

We conducted a technical workshop to generate missing AR-specific knowledge. Here, participants were able to investigate main hardware-related aspects of AR devices. However, some aspects could not be examined in the given time frame. The question therefore arises how technical expert

knowledge can efficiently be integrated in early stages of requirements engineering to obtain required knowledge about AR-specific aspects (4a). Alternatively, how requirement engineering methods can be applied in such a way that participants can efficiently generate this knowledge without delaying the requirements engineering process (4b).

Within our requirements engineering sessions, we observed that quality requirements, especially with regard to environmental aspects, took an essential role. Thus, special environmental demands of the desired AR application need to be defined through appropriate requirements engineering techniques. These aspects are essential since they have an influence on all phases of the software development process [68]. Thus, approaches are needed to systematically integrate aspects of quality and environmental requirements into methods of requirements engineering (4c).

As Martínez et al. [85] describe, the current AR landscape is characterized by a majority of small applications tailored to specific, well-defined use cases, which are applied particularly in the industrial domain. However, it is essential in every domain to involve domain experts in early phases to generate domain-specific requirements. Within the realm of AR, the advanced technical complexity of the novel technology complicates the integration of domain experts. Thus, techniques are needed that enable the integration of domain experts by reducing the technical hurdles in order to be able to collect domain-specific requirements (4d), e.g. through simplifying the definition of AR artifacts (e.g., [16]) to explore and generate domain-specific requirements.

Research Agenda

- 4a. How can technical expert knowledge be efficiently integrated into early stages of requirements engineering to obtain required knowledge about AR-specific aspects?
- 4b. How can participants be supported by engineering methods to efficiently generate missing knowledge about AR-specific requirements?
- 4c. How can quality and environmental demands be systematically integrated into requirements engineering techniques?
- 4d. How can the technical hurdle in AR be reduced so that domain experts can efficiently be integrated into requirements engineering activities?

5 UI DESIGN

Within the software development process, UI design aims at an effective interaction between human and machine and should be tailored to the skills, experiences, and expectations of anticipating users [17]. UI design has high relevance in AR development and has been stated as a major challenge [39]. The choice of a UI concept has a significant impact on the quality of an AR application. As a result, recent research focuses on how different interaction modalities and visualization modes influence the efficiency and performance, e.g. of industrial AR applications (e.g., [62, 105]).

Within AR research, common UI engineering techniques are usually adopted to AR, like sketching (e.g., [43, 76]) or prototyping (e.g., [27]). In this section, we explore UI engineering activities and highlight their application within the AR domain. In line with Buxton [21], we consider sketching and prototyping as two independent activities, since they serve different purposes within the design process. Sketches are created in early ideation stages, whereas prototypes tend to be used in later stages when initial ideas have been sharpened. In Section 5.1, we discuss the application of sketching in early design phases. We further provide an overview of prototyping in the context of AR in Section 5.2 by combining current research with our hands-on experience. In Section 5.3, we focus on storyboards and explore the integration of AR-specific artifacts. Our findings and resulting research questions are summarized in Section 5.4, where a research agenda for the UI design in AR development is formulated.

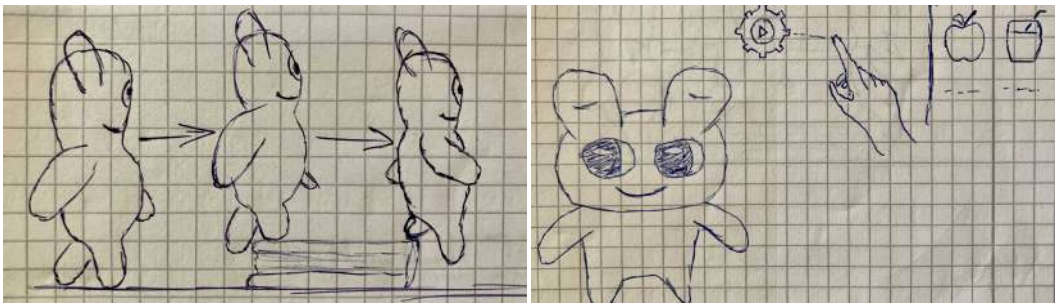

Fig. 1. Sketch of the virtual pet interacting with the physical environment (left) and a proposed gesture interaction idea for feeding (right)

5.1 Sketching

Sketching is an activity in which first design concepts are developed through free-hand drawings. In line with Fish and Scrivener [36], we see sketching as an untidy representation to visualize ideas. According to Buxton [21], efficient sketches should comprise the following basic characteristics: They should be quick to create, have a low level of detail and be ambiguous to encourage discussion. Software engineers use sketches to achieve a common understanding and avoid strong formalization, which is rather seen as a hindrance [102]. In UI design, sketches can also assist in the definition of interaction concepts, e.g. by simplifying gesture descriptions, which can be represented in sketches and collaboratively defined and quickly exchanged within design sessions (e.g., [58]).

There are several approaches in research where sketching is used for UI design when developing AR applications. Hagbi et al. [49] apply pen and paper sketches within content creation for AR games. They define a visual language that enables the generation of virtual content from sketches. Langlotz et al. [76] present an authoring tool for mobile AR applications, allowing users to freehand sketch 3D primitives. Furthermore, Gasques et al. [43] apply sketching within *PintAR*, a tool in which sketches are produced on a tablet and then transferred to a HMD along with predefined interactions.

Within all our mentioned AR projects, sketches were produced in early design phases in order to discuss initial design ideas, especially regarding the visual appearance and desired system behavior. For example, within our virtual pet project, we first illustrated our expectations and visions of realistic animations of the virtual pet within the physical environment, as shown in Figure 1. In a later design iteration, the sketches became more detailed, as we added specific gesture interactions to our sketches. Since AR technology offers many possibilities for the appearance and behavior of virtual objects in the real world, we found sketches to be a useful tool to rapidly explore as many ideas and concepts as possible. Assuming that several stakeholders have many different visions of how virtual and real objects should be combined, sketches help to visualize them.

However, we did not consider the technical feasibility of ideas in our sketches at first, in order not to limit the process. Since AR applications may include ambitious and visionary settings with complex user interfaces, we consider the evaluation of technical feasibility by experts to be convenient in order to streamline the sketching process. Finally, the created sketches served as a suitable blueprint for the initial modeling of 3D content, which supported the design process.

5.2 Prototyping

In software development, prototyping assists in generating and capturing system requirements, examining software artifacts, and developing user interfaces [106]. In line with de Sá and Churchill [27],

we define probing, concept validation, feature validation, usability testing, and User Experience (UX) evaluation as the main use cases of prototyping within the software development cycle. Prototyping techniques can be divided into low-, mixed-, and high-fidelity techniques. Low-fidelity prototypes usually concentrate on broad design ideas and aid in evaluating many different design concepts, since they can be produced rapidly [18]. For example, mock-ups or pen and paper prototypes have no emotional attachment, and thus can be exchanged quickly, which corresponds to a cost-effective iterative design [61]. In contrast, high-fidelity prototypes focus on interactivity and functionality. They are usually digital and contain a realistic representation of the visual design concept [86]. Since these prototypes are similar in look and feel to the final product, they are especially useful for a detailed evaluation of core design artifacts [8]. High-fidelity prototypes often refine and substantiate existing low-fidelity prototypes [112]. Mixed-fidelity prototyping (e.g., video prototyping) is a hybrid version of the other two [86].

There has been little research on whether conventional prototyping techniques are suited for AR software development so far. Stated special characteristics of AR applications, like novel interaction techniques, environmental factors, or missing usability standards, might lead to special requirements for software prototyping. As a result, many challenges regarding AR prototyping are described within the recent literature. For example, every AR device group features different characteristics, leading to special requirements depending on the device, like interaction metaphors or device-specific restrictions that need to be replicated in AR prototyping [66]. Another challenge is the relevance of the real environment. This issue already occurs in low-fidelity prototypes, like paper prototyping, which seems to be more suitable for static physical backgrounds [77]. Alce et al. [4] present a wizard-of-oz-approach that aids in testing the visualization of 2D data, tactile, and auditive feedback. Although this specific approach is not capable of simulating scenarios involving real-time tracking, we assume this prototyping technique to be beneficial due to its ease of adaptability. As a fact, Freitas et al. [37] found wizard-of-oz prototyping to be one of the most applied prototyping techniques in AR development. Keating et al. [68] argue for the necessity to prototype physical environments in order to prevent the in situ evaluation of AR prototypes, which is particularly difficult, since the foundation of AR software is usually the location specific camera view [68]. Iterative software development becomes laborious when every change requires leaving the office for a new field trip. Although environmental prototypes might prevent field visits, the prototyping process becomes more expensive and time consuming. In this context, *C-Space* [107] was designed as a support system that provides spatial modeling and is proposed as a prototyping tool for indoor AR applications.

Due to the lack of standards, UI concepts must be defined in close cooperation with end users to ensure adequate usability. Kang et al. [65] developed several low-fidelity prototypes as part of a participatory design process involving children and teachers to gather design ideas for an AR learning application. Others dealt with the participation of stakeholders (e.g., end users) in prototyping sessions. Burova et al. [20] investigated how and at what position virtual content should be presented to end users. They used VR with gaze tracking to prototype and evaluate AR content and were afterwards able to foster design decisions. Damala and Stojanovic [25] emphasize that multiple stakeholders such as AR experts, domain experts, and end users need to participate in a collaborative design process. They point out that different types of stakeholders need to make decisions about what kind of AR content needs to be provided and how to present the content on the AR display.

Regarding the roles within the AR software development process, we suspect and argue for a role shift, especially when it comes to digital AR prototyping, in line with Börsting and Gruhn [15]. In conventional software projects, low-fidelity prototypes like physical prototypes, pen and paper prototypes, or digital mockups are often build by UI designers. Software developers are then first

Fig. 2. Physical prototype and HMD representation

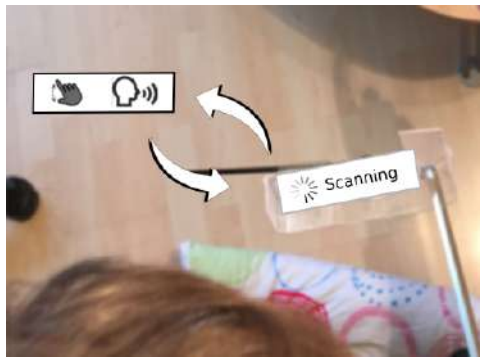

Fig. 3. Representation of UI changes

involved when functionality should be added to a prototype. In AR development, UI designers are able to create analog prototypes like sketches, but even the construction of simple digital prototypes (e.g., video prototypes or mockups) requires technical knowledge since there is a lack of tool support for creating digital AR prototypes. Therefore, software developers need to be involved in early design activities, resulting in changes to the development process.

To further investigate stated challenges and to evaluate different prototyping techniques in the realm of AR, we examined analog and digital low-fidelity prototyping techniques, in form of a physical prototype and a video prototype. We focussed on low-fidelity techniques since they are easy to create and aid in generating and evaluating many different design solutions [8]. Although our constructed prototypes do not cover the whole spectrum of potential prototyping techniques, we assume our findings to be transferrable to other techniques.

As an analog low-fidelity prototyping technique, we chose to built a physical prototype using Lego bricks for our virtual pet application (see Figure 2). This prototype served as an analog representation of the virtual pet that should later be integrated into the final AR software. To simulate an HMD, we used plastic glasses to which an allen wrench was attached. The short side of the wrench was placed within the user's field of view. We glued a tray to the allen wrench serving as a surface for small paper strips that represent UI artifacts. Additionally, a Lego antenna was centered in the field of view, representing the cursor of an HMD.

Using this physical prototype, we conducted a qualitative user-based evaluation in order to explore design and interaction concepts. Within the user evaluation, state changes were displayed by manually moving the physical prototype through the environment or rearranging arms and legs. To mimic UI changes, paper strips within the tray were exchanged (see Figure 3). We examined the prototype construction and evaluation process through observation techniques, especialy regarding the general understanding of AR systems based on HMDs, the exploration of basic interaction concepts of the specific software, and the design evaluation of UI elements. As a result, all participants stated that they gained a basic understanding of the AR system. Additionally, participants expressed that the prototypical plastic glasses helped in understanding how to interact with HMDs. In contrast, we found the manual simulation of realistic interactions as particularly challenging. The exchanging of paper strips and the rearrangement of the physical object within the environment were carried out by the same researcher, which made it difficult to display interactions

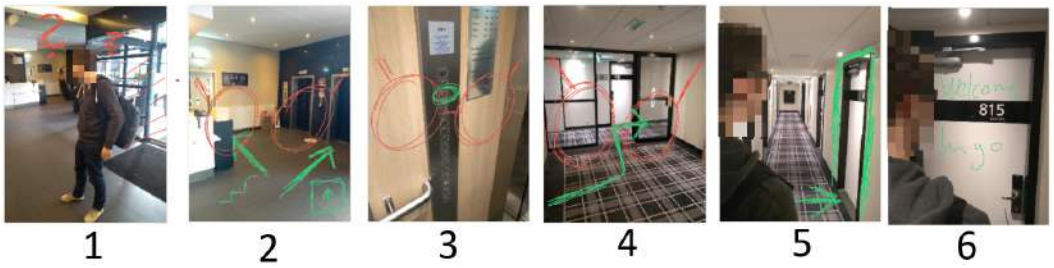

Fig. 4. Storyboard for an AR indoor navigation app (AR overlay in green, additional annotations in red)

in real-time. As Liu and Khooshabeh [81] state, simulating realistic interactions within analog prototypes is laborious, since many researchers need to be involved.

In order to investigate digital low-fidelity prototyping techniques for AR application development, we constructed and qualitative evaluated a video prototype for the virtual pet application. Here, a captured physical environment from the user's perspective served as a foundation. A representation of the virtual pet was modeled using basic shapes and integrated as a digital overlay. Interactions were simulated by the researcher, who performed gestures to visualize interaction concepts. In contrast to the analog prototype, constructing the video prototype was found to be time-consuming. Displaying interactions within the video prototype was especially laborious, since the pet's reactions had to be adapted frame by frame in order to simulate interactions. To shorten the prototype construction time, special expertise in video editing would be necessary. However, participants evaluated that the video prototype demonstrated the essential functions of the software and the interaction concept rapidly, but that the prototype was less suited for a basic understanding of the AR technology. This confirms the findings of de Sá and Churchill [27], who claim that video prototypes are not well suited for exploring new concepts, but good for feature and interaction validation.

5.3 Storyboarding

A storyboard is a graphical representation of a narrative originating from film and TV productions [51]. In the context of software engineering, we rely on the definition by Greenberg et al. [45], that storyboards are often used to illustrate user interaction through a sequence of UI sketches representing a dialog flow, but might also include the application context and environment.

Within our mentioned AR projects, we explored storyboarding considering AR-specific details. We focused on the differentiation between virtual objects and the physical environment. Explanatory annotations, which are not meant to be part of the desired UI, but provide additional information for the storyboard, were also considered. Additionally, the representation of the AR device as well as movements and interactions were examined.

First, we created an UI concept for a potential AR indoor navigation app within a storyboarding workshop. In contrast to our mentioned AR applications (see Section 2), this use case was not considered further, as the workshop session focused exclusively on storyboarding. Here, photos were taken as a foundation and edited subsequently (see Figure 4). Our storyboard included different perspectives to visualize the future application's use. This comprises a third person perspective, focussing on the user carrying the AR device (represented by sunglasses), a first-person perspective from the user's field of view, focussing on the information visualization, as well as an over-the-shoulder perspective as a combination of the other two perspectives. Here, the distinction between virtual objects and the physical world was already realized through the photographic foundation.

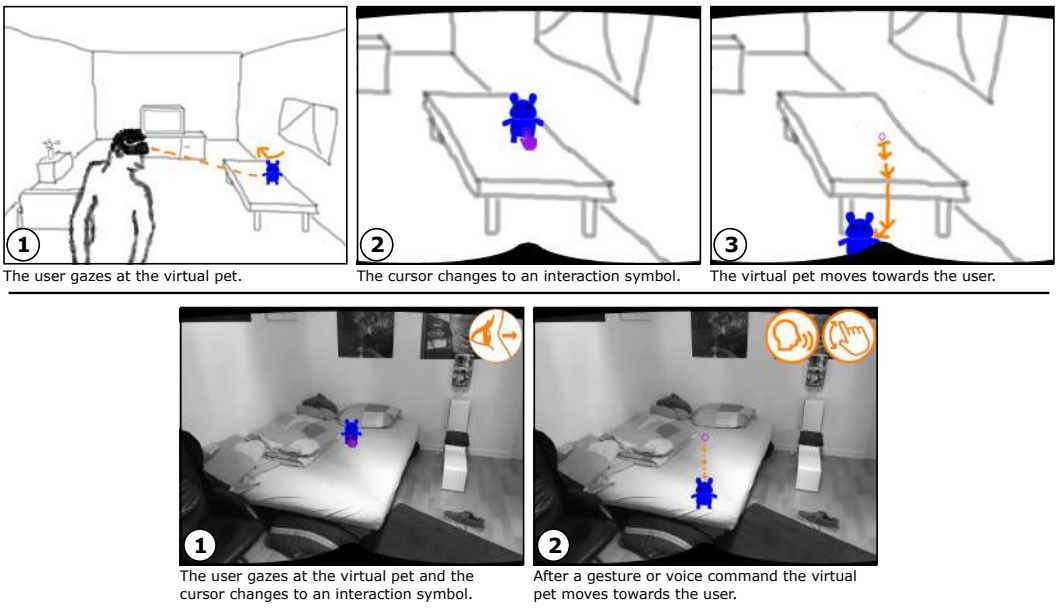

Fig. 5. Storyboards for the virtual pet application

Virtual objects were drawn in a bright green to stand out from the background. We added additional annotations to the storyboard in red, e.g. to express the intention of the application use or to visualize that certain objects are only visible when the user carries an AR device.

Within our virtual pet project, we further investigated storyboarding by comparing sketched and photo-based storyboards. When creating sketched storyboards, we considered a low level of detail and textual description (see [110]) as well as different colors for highlighting virtual objects, movements and interactions. As shown in Figure 5, we decided to draw the physical environment in a light grey, the virtual object in blue, movements and interactions in orange, and the HMD cursor in purple. We decided to focus on a first person view within our storyboards to conceptualize the application from the user's perspective. The third-person-view was only used to initially introduce the application context. In a further development iteration, we extended our storyboards using photographs and added annotations to visualize interaction (see Figure 5). We decided to use black and white photos to represent the physical environment without distracting from virtual overlays. Here, annotations represent common interaction possibilities like gaze, gestures or voice commands, illustrating the virtual pet moving towards the users after a gesture or voice command is performed. We found the annotations to allow a more explicit statement about which form of interaction should be implemented at which time. Our photographs served as an easy and quick representation of the real world without requiring drawing skills. Nevertheless, it is conceivable that this process would become more complex as soon as the physical environment has to be particularly prepared. Furthermore, with photo-based storyboards, the tendency is higher that details in the background distract from actual UI elements, which of course strongly depends on the photographed environment. We found using black and white photos to be a possible solution to this issue.

While creating storyboards, we found the graphical representation including color and symbol annotations as suitable for explaining application characteristics and behavior. Adding photos to storyboards assists in visualizing interactions and movements within a physical environment.

Compared to sketches, we found storyboards to require more effort, since more images need to be produced, which is multiplied by the number of states and processes to be considered. To address this issue, Greenberg et al. [45] propose to branch storyboards, since users may follow different paths in UIs. However, the adoption of this approach to AR, with its multiple modalities and varying real world conditions, has yet to be investigated.

5.4 Findings

Within this section, we examined common UI design activities regarding their application within AR application development, including sketching, prototyping, and storyboards.

During the sketching sessions for our AR projects, AR-specific aspects, like interacting with the physical world, played a major role. We believe that in early design phases of AR applications, sketching is one of the most relevant activities, as many options for the appearance and behavior of virtual objects in the real world can be quickly demonstrated. Since the limits of the AR technology cannot be estimated by most stakeholders, we assume that the technical feasibility is neglected in many sketching sessions. Here, it should be examined whether the integration of AR experts is helpful for streamlining the sketching process (5a).

As part of the literature review on AR prototyping issues, we found the choice of the targeted AR device to be crucial for the selection of appropriate prototyping techniques. For physical prototyping, a useful representation of the device needs to be built in order to display realistic interactions. This can be laborious and time-consuming, depending on the prototyping technique and the device. Besides AR devices, the physical environment is an essential aspect of AR systems and therefore an important part in prototyping. In some cases of analog prototyping, the physical environment has to be prototyped as well in order to simulate realistic scenarios. In line with Farinazzo Martins et al. [34], software engineers should not only focus on functional and quality requirements, but also determine environmental requirements that need to be faced. Therefore, environmental aspects of the AR software need to be validated when selecting a prototyping technique. The question of how devices and physical environments can be efficiently integrated or simulated in AR-prototyping should be further investigated (5b, 5c).

Additionally, recent research suggests that digital prototypes have a certain appeal when creating AR applications [43, 79]. However, AR development capabilities would currently be necessary, leading to a shift in either skill requirements for UI designers or a role shift within the development process, because software engineers are now required for prototyping. We assume this to be confirmed in further digital prototyping techniques, like mockups or wireframes, especially if they contain minor functionality. Research is needed supporting the creation of digital prototypes by designers with no programming skills (5d). First steps have been made in this direction (e.g., [15]).

Within our mentioned AR projects, we found storyboarding to be a useful technique to visualize and communicate UI artifacts. We decided to differentiate AR-specific elements by colors and represent interactions through annotations. We found the simplified display of the spatial distribution of UI elements within storyboards to be a benefit for AR UI design. In future research, the comparison of AR storyboards for HMDs and mobile devices would be an interesting research topic. A formulation of best practices that either extend across devices or are applicable to a specific device group should be aimed at. Here, AR-typical multimodal interaction concepts and changing environmental conditions of the physical world should be considered (5e).

Research Agenda

5a. How can knowledge of AR experts be integrated into the UI design of AR applications?
5b. How can devices be efficiently integrated or simulated in AR prototyping?
5c. How can physical environments be efficiently integrated or simulated in AR-prototyping?
5d. How can the digital AR prototyping process be supported to not require advanced technical skills?
5e. How can best practices for AR storyboarding be formulated considering different devices and environmental conditions?

6 SOFTWARE MODELING AND IMPLEMENTATION

During implementation, the engineering team creates an executable version of the software based on requirements and design decisions. This phase includes the use of specific programming languages and possible adaptation to and integration of standard components to meet the requirements [106]. According to Sommerville [106], there are several implementation principles that should be considered when implementing software. Among them is the potential reuse of software (components), meaning that software should be implemented in a way that it can be transferred to other applications. Most AR applications, however, are currently tailored to a very specific use case and its components cannot be reused in other applications [85].

In this section, we highlight challenges in the implementation of AR applications and differences in comparison to conventional software projects. First, we highlight methods for visualizing an overview of the desired system and how diagrams can be applied for representing AR system artifacts and user interaction in Section 6.1. We then discuss existing tools and frameworks that assist the implementation of AR applications in Section 6.2. We further focus on current approaches to localize and register physical objects within the real world in Section 6.3 and on debugging AR software in Section 6.4. Finally, we summarize the present section with a concluding list of research questions in Section 6.5.

6.1 Software Modeling

Within the scope of our work, we addressed the creation of system overviews, which especially serves the visualization of system components in order to generate a common understanding of the application. Thus, before the actual implementation of our virtual pet application, we aimed at presenting the results of previous phases in an abstract overview to support the communication with developers. The first step was to describe the semantics and behavior of the desired application without already anticipating details. We applied a visualization method from Schmalstieg and Höllerer [100], which uses a theater metaphor to represent and relate essential AR artifacts (see Figure 6). Based on hardware that provides user input and output, this approach defines a *Story* (application logic) driven by *Interactions* and influencing *Actors* on *Stages*. Furthermore, *External Factors*, such as lighting conditions, which play a fundamental role in AR applications, can be considered. Here, all artifacts that represent application content are seen as *Actors*, comprising virtual objects as well as sounds or video clips. Physical artifacts registered by the application and used as a foundation for representing virtual objects (e.g., flat surfaces) are specified as *Stages*.

As shown in Figure 6, we applied the theater metaphor to represent the generated requirements of our virtual pet application in terms of *Interactions*, *Stories*, *Actors* and *Stages*. Here, interaction forms of the Microsoft HoloLens like gaze, gestures, and voice were defined as modalities to drive the application logic. For our application, defining the *Story* was particularly difficult, since events do not follow a chronological order. The *Story* in our application rather consists of a non-linear

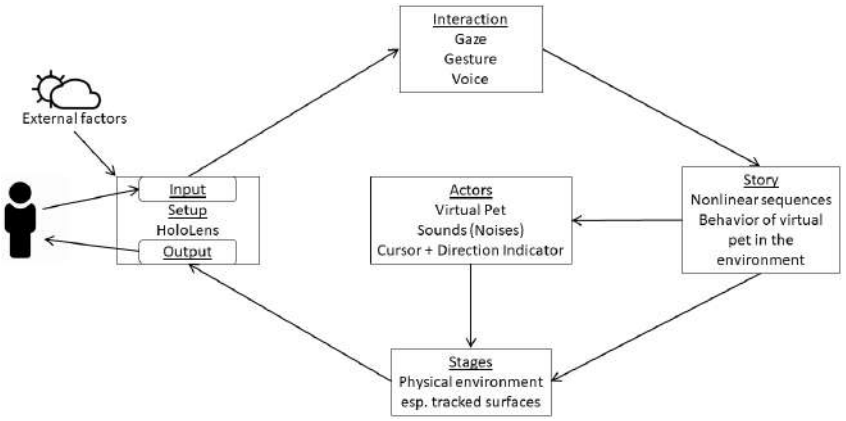

Fig. 6. System overview of the virtual pet application

sequence of events. For this purpose, Schmalstieg and Höllerer [100] propose to represent non-linear events as state diagrams, similar to other event-based systems. The virtual pet itself and the sounds representing its noises serve as *Actors* within the overview. The cursor of the HMD is added to the *Actors*, since its visual representation varies depending on the application state. The *Stages* within our theater metaphor comprise the registered surfaces located in the physical world on which the virtual pet is supposed to move.

The adoption of the approach from Schmalstieg and Höllerer [100] served as a tool to rapidly create an overview of the desired AR system including specific artifacts. We observed that this overview simplified the collaboration between designers and developers by rapidly creating a common understanding of the intended system.

After the high-level consideration of general system components, we explored the practicality of diagram-based modeling in the context of AR implementation. In our virtual pet application, we first focussed on use case diagrams to further specify user interactions. Although these diagrams are usually created in earlier development phases, we attributed them special relevance for implementation, especially regarding the interrelation between actors and actions. Here, we considered use cases in which users directly interfere with the system. However, it is in the nature of our application that not every event is triggered by user interaction, but that the virtual pet also acts autonomously, including independent movements within the physical environment. We decided that, contrary to the classical application of use case diagrams, it is useful to include these use cases as well, especially if the user is supposed to react to autonomous actions. Thus, our virtual pet was considered as an individual actor within our use case diagrams.

We further considered state machines to be suitable for non-linear AR applications. These state diagrams represent a form of event-driven modeling [50] and consist of system states and events that trigger transitions from one state to another with regard to hierarchical states and concurrency. A state diagram representing our virtual pet application is shown in Figure 7. Here, the localization of the physical world is defined as the initial state. Next, the virtual pet appears at a suitable location within the registered environment. All further system behavior is only executed as long as the virtual pet is visible (state: *Virtual pet visible*). In addition, a *default* state was defined, implying autonomous behavior of the virtual object and triggered when the user does not interact with the virtual pet. As shown in the diagram, this system state is left as soon as the user gazes at the virtual pet. Further states of the virtual pet are triggered by the user's interactions. The states *eating* and

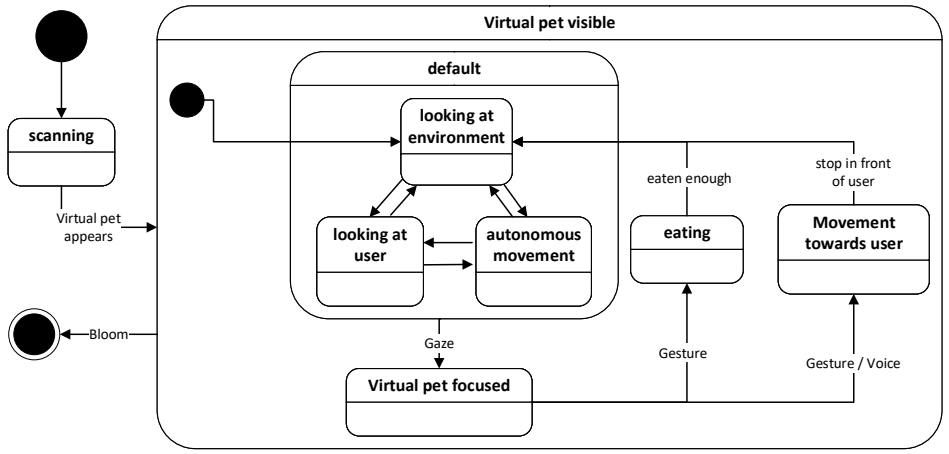

Fig. 7. Unified Modeling Language (UML) state diagram representing the overall system behavior

movement towards user are defined and triggered by either gestures or voice commands. As soon as the user performs the gesture *Bloom*, the end state of the application is triggered.

We found diagram-based modeling to be a useful vehicle for generating a shared understanding of the AR application and uncovering connections between system artifacts. However, the benefit of such diagrams should be considered in the context of the time required to construct them.

6.2 Frameworks and Tools

In AR research, the need for supporting frameworks is stated in various publications (e.g., [83, 92]), since implementing software that includes 3D objects in an 3D environment often results in a laborious and time-consuming process, due to non-standardized hardware and quite primitive development tools [92]. Thus, several approaches have been presented to simplify the implementation process of AR systems. For example, Piekarski and Thomas [92] define *Tibnith-evo5*, a software architecture for building mixed reality applications. To simplify the software development for programmers, a data flow methodology with an object-oriented design is used, allowing distributed programming, persistent storage, and runtime configuration. Here, a building blocks approach combines graphical function blocks and coded scripts, allowing to model AR applications without programming skills. *Studierstube* [100] focussed on simplifying 3D modeling, proposing an authoring tool that takes place within the 3D environment. Programmers and experts are enabled to collaboratively model 3D objects using external devices like tablets and pens. Additionally, Conway et al. [24] propose an approach to use technical documentation to generate virtual objects and augmented experiences. Here, Computer-Aided Design (CAD) models of technical objects are analyzed and automatically transformed into augmented artifacts. Furthermore, Müller et al. [87] present an approach in which the modeling of virtual objects and associated animations is based on the motion capturing of their real-world representations.

Although different concepts for simplified software development exist within the literature, these are underrepresented in common tools for AR software development. As Gandy and MacIntyre [40] state, many frameworks within the literature focus on prototyping instead of deployable software. However, current commercial frameworks, which generally aim at deployable software, pose further challenges in the AR software development process.

The current landscape of mobile AR frameworks is shaped by Google's ARCore[1], Apple's ARKit[2] and Vuforia[3]. Game development environments such as Unity[4] and Unreal Engine[5] have established themselves as Integrated Development Environments (IDEs) for AR software development. These development environments enable hybrid application development by deploying applications to various operating systems. Nevertheless, these development environments differ strongly from conventional IDEs, especially in terms of structure and usability. AR developers must adapt to the IDE required by the chosen framework, which is a contradiction to the statement of MacIntyre et al. [83] that the workflow of tools should be tailored to the medium and the target group. Current IDEs for AR development might require long training periods, which leads to a slower and less efficient development process.

Within our AR projects, we observed that even experienced programmers needed a long training period to master the structure and handling of these IDEs. In addition, further obstacles become apparent when using current IDEs for AR software development. As already stated by Seichter et al. [101], the relevance of the physical world should be considered in AR development environments in order to simplify the implementation process. Currently, a few initial simulation approaches emerge in AR development, such as *Unity MARS*[6], which is capable of simulating environmental and sensor data, or *DistanciAR* (see [113]), which enables the capturing of real-world environments and the authoring of AR applications within those environments. Nevertheless, these promising approaches are not yet established or tested for effectiveness. It can be assumed that applications still need to be repeatedly deployed on external devices if the application is to be debugged in the context of the physical world, leading to a laborious development process. Another possible approach to consider the physical environment within the development process might be to transfer development activities into the augmented reality, such as the prototypical implementation and evaluation of interaction concepts, which are subsequently finalized in efficient desktop environments (see [16]). This simplifies the optimization of the software's usability, since the modeling is performed from the user's point-of-view [3]. This further reduces the cognitive load on software designers, as no cognitive transfer from a 2D desktop environment to the augmented 3D environment is required [78]. However, process models would be needed that allow these artifacts to be efficiently exchanged between actors. Research should further address the relevance of the physical world by investigating how the real environment can efficiently be integrated into development frameworks. As mentioned, an alternative approach is to examine how development activities can be adapted for an application within the augmented reality and resulting artifacts be efficiently exchanged between actors.

6.3 Localization and Registration

The enhancement of the physical world with virtual content is the core aspect of AR. Tracking the real world enables, for example, the analysis of the context of use, so that UI elements can be arranged and designed in relation to the physical environment (see [28]). In AR technology, however, the detection of physical artifacts for their inclusion in AR software development is essential and remains a key research topic [11]. The localization and registration of physical objects has to be considered throughout the entire development process and imposes several challenges on developers, particularly regarding its implementation. In AR application development, many

[1]https://developers.google.com/ar/discover/
[2]https://developer.apple.com/augmented-reality/arkit/
[3]https://developer.vuforia.com/
[4]https://unity3d.com/de
[5]https://www.unrealengine.com/
[6]https://unity.com/de/products/unity-mars

approaches have been described to simplify tracking purposes. These approaches can be divided into *marker-based* and *marker-less* approaches.

Marker-based approaches are based on analog or digital markers. Analog markers include visual markers such as QR codes or image targets. Digital markers include bluetooth beacons, RFID, and NFC tags as well as geo tags, which can be accessed via GPS sensors. Marker-based approaches allow a robust registration of physical objects. The localization requires only little computing effort and thus supports a wide range of devices. Furthermore, the implementation of marker-based object registration is simplified, since frameworks like Vuforia aid in the definition and assignment of visual markers to the desired logic. Location-based approaches are based on GPS data in combination with the device's camera image. Here, existing databases of geo-locations are used to compare the camera image with stored image material. Thus, they are mainly used for the localization of buildings and points of interest in outdoor applications. Geo-based approaches also conserve resources, as complex calculations are usually not carried out on the device itself (see [109]). Since physical objects are represented by analog or digital markers in marker-based approaches, an actual knowledge about the real world is not generated. Thus, these approaches are limited in their flexibility, since all physical objects must be registered beforehand. The integration of physical elements is therefore restricted to predefined objects.

Marker-less approaches are usually implemented by image processing, providing an understanding of the physical environment and the position of the user in that environment. Here, the classification of physical objects is usually accomplished by the application of common algorithms, such as SLAM [7] or YOLO [95]. These approaches allow flexible object recognition, since no prior registration of specific objects is necessary. However, many algorithms require the training of models to classify objects. The immediate classification of physical objects requires a high computing power, which cannot be guaranteed by current devices [85]. Outsourcing the calculation to the cloud is conceivable, but requires a suitable network connection. In addition, marker-less approaches are prone to errors – current algorithms cannot provide the high localization accuracy required by most AR applications [85].

In AR application development, the choice of a suitable tracking method usually depends on the target device. HMDs allow a detailed mapping of the physical world, where the real world is analyzed and provided by an infrared mesh. However, the classification of physical objects is only possible by adding algorithms for object classification and the training of models. Mobile AR devices are usually limited to the detection of horizontal and vertical surfaces, due to lack of sensors and efficient algorithms. For example, Google's ARCore framework determines visual feature points within the camera image. From these feature points, clusters provided as planes within the SDK are generated. As shown in Figure 8, planes can be differentiated and applied within the application, e.g. as a basis for a game field like in our AR adaptation of Pong. Here, the plane's dimensions are used to fade in game elements and thus integrate the physical world into the gaming experience. ARCore distinguishes planes of different levels, which are visualized by alternating colors.

The integration of tracking methods differs fundamentally depending on which localization technique is to be used. The rather simple implementation of visual markers requires the prior registration of physical objects, but the effort and required expert knowledge is rather low. The implementation of marker-less methods is higher in complexity and effort. Although the localization of surfaces is still relatively easy to implement using supporting frameworks, as soon as a real object classification is to be implemented, expert knowledge from the field of Machine Learning (ML) as well as the training of corresponding models is necessary.

When implementing AR applications, software developers need intensive knowledge of common methods for localizing the physical world. Depending on the procedure and framework, the implementation differs in complexity and effort. The wide range of devices and localization methods

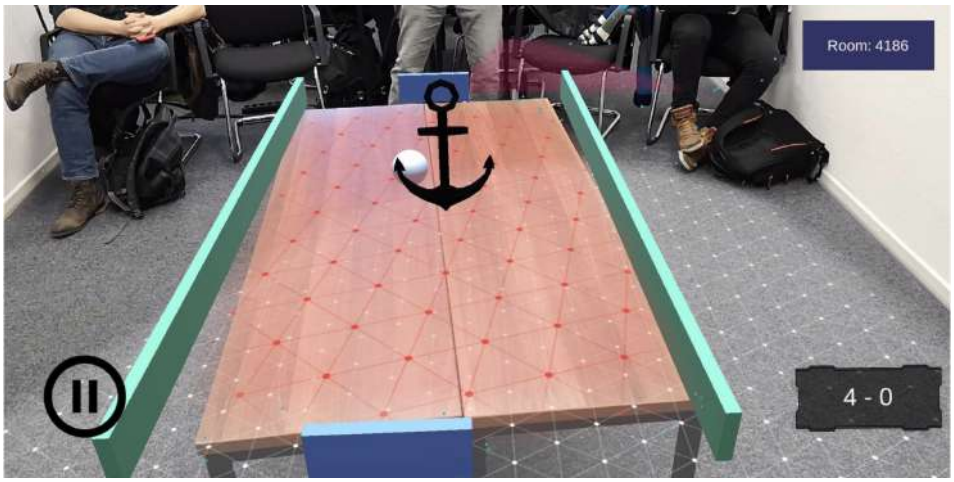

Fig. 8. Plane utilized as a game field within the AR Pong adaption

forces software developers to adapt to methods and frameworks. It needs to be researched how methods can be standardized, so they can be used across devices and frameworks. Furthermore, current ML algorithms are too complex to apply to AR applications due to the required computing power and the extensive training of models, although with the ever increasing processing power of mobile devices and the availability of ML models optimized for mobile devices (e.g., based on MobileNetV2 [98]). In addition, they lack localization accuracy, especially in unstable environmental settings (e.g., varying light conditions) [85]. Approaches that either define AR-specialized classification algorithms for current devices or that allow the efficient outsourcing of real-time computation, e.g. to cloud services, are needed.

6.4 Debugging

When implementing software, developers often need to check code blocks for errors and remove or fix faulty functionality. The localization and correction of errors in a software product is commonly referred to as *debugging* [108].

Within our mentioned AR projects, various debugging approaches based on different AR devices were examined, particularly regarding provided device emulators. In common development environments, emulators are usually provided to rapidly debug software without the necessity of actually deploying software to an AR device. We examined smartphone emulators while developing the AR Pong application as well as the Microsoft HoloLens emulator within the virtual pet development. These emulators turned out to be useful to quickly debug UI elements, but were limited in their application, so that not every functionality could be tested. As a result, actual AR devices were still needed, e.g. to verify the integration of virtual objects within the physical environment. Furthermore, the debugging of network and synchronization functionalities in our AR Pong application was not covered by the emulators, so that these aspects had to be tested on actual devices, leading to a laborious debugging process. The fact that multiple devices are needed to debug synchronization functionality as well as the still high cost for AR devices (especially in the HMD segment) might even lead to features impossible to debug efficiently when the required number of devices is simply not available.

In sum, we found debugging activities in our AR projects to be laborious despite provided device emulators. Advanced functionality, like the synchronization of information between devices, still

had to be debugged on actual devices, leading to a time-consuming process. In addition, debugging functionality concerning the physical world turned out to be difficult, since robust approaches for the simulation of the real environment are scarce in common tools (see Section 6.2). Thus, testing the localization of physical objects still had to be performed on actual devices, including the laborious preparation of the physical world in order to debug certain functionality.

6.5 Findings

In this section, we examined the software implementation phase by investigating individual techniques. We emphasized that the development of AR applications differs in various aspects from the development of conventional software systems.

Within our AR projects, we found that the high-level visualization of system components can assist the collaboration of designers and developers by quickly creating a common understanding of the desired system, which can be essential in an AR software development usually characterized by inexperience and uncertainty. An additional representation of non-linear behavior of the application can be achieved by state diagrams, which allowed to link state transitions to AR interactions within our virtual pet prototype. Nevertheless, it must be evaluated whether the expenditure of creating formal models justifies the final value for the AR software development process (6a).

In addition, AR-specific frameworks have been defined in the scientific field, but are rarely applied in current AR application development, because many of these tools target prototyping rather than deployable software, and yet usually require technical expertise (6b). The current landscape of commercial tools for the development of AR applications is characterized by tools from game development or development of mobile applications. A crucial issue of current frameworks is that the relevance of the physical world is not fully acknowledged yet, since the efficiency of ermerging simulation approaches is still questionable. This led to a complex debugging process that required continuous build and deployment activities within our AR projects. Thus, approaches are either needed that enable an efficient representation of the physical world within the development environment (6c) or approaches that apply development activities within the augmented reality need to be formulated, while considering the efficient exchange of created artifacts within the development process (6d).

There are many different tracking methods used in AR application development, which can be differentiated as marker-based and marker-less approaches. Marker-based approaches are rather easy to implement, but are limited to predefined objects. Marker-less approaches aim to generate an actual knowledge about the physical world, but have high demands on computing power and need to be extended with ML algorithms for object classification. AR-specific tracking algorithms are needed, which allow an efficient object classification, by either considering the limited computing power of current devices or outsourcing complex operations, e.g. to cloud services (6e). Additionally, we found the wide range of different localization methods to require a broad knowledge about individual methods and the adaption to these methods and frameworks. Localization methods must be generalized so that they can be applied across devices and frameworks in order to reduce the training time and contribute to an efficient implementation process (6f).

Research Agenda

6a. How can the value of modeling activities in AR projects be evaluated in the context of the effort required to create them?
6b. How can prototyping concepts from scientific sources be adapted to toolkits for deployable AR software?
6c. How can physical objects and environments be integrated or simulated in development environments?
6d. How can development activities be transferred to AR and directly performed there?
6e. How can AR-specific algorithms be defined that meet the demand for tracking and registration accuracy under consideration of limited computing power?
6f. How can tracking methods be generalized so that they can be applied across devices and frameworks?

7 TESTING

Testing is an essential activity for the quality assurance of software. Here, the comparison of a target with an actual behavior of a test object determines whether the test object fulfills required properties [108].

In the following section, we first highlight test types within the software life cycle in Section 7.1 and indicate how underlying process artifacts can be transferred to AR testing. We then address interaction testing and test automation in Section 7.2. In Section 7.3 we give an insight into usability testing in the context of AR by discussing different approaches and linking them to our own experiences. Finally, we present findings and resulting research questions in Section 7.4.

7.1 Testing within the software life cycle

Within the software life cycle, test activities ranging from evaluating small components to complete systems are carried out. We consider test activities following Boehm's V-model [13] and discuss their application to AR development. Since testing activities included in this established development model are also considered in current frameworks, we used this model solely to structure this section.

Unit tests. In unit tests (also known as component, module, or class tests), smallest software units are tested isolated from other components to prevent external influences. Developers usually apply test drivers or stubs to check the correctness of test objects. In Test-Driven Development (TDD), software developers write unit tests before implementing components, enabling clean and flexible code [84]. In AR applications, a large part of the code base consists of the implementation of interaction concepts. Traditional UIs are typically built using interaction toolkits that provide widgets with integrated event handlers. A corresponding unit test simulates the input and evaluates the results of event callbacks. However, in AR development there are no common UI toolkits with event handling, resulting in a lack of tools for testers to automatically test device input. Thus, device input such as gestures need to be executed manually to evaluate them.

However, unit testing is already being considered in related research areas. Bierbaum et al. [10] modify the conventional unit testing for the application within VR development. They use recorded inputs and define states in a VR application scenario, which are subsequently compared with the test results. Nevertheless, it remains unclear how unit tests can effectively be integrated into AR development.

Integration tests. After individual software units have been composed into comprehensive modules and subsystems, the integration test evaluates the interoperability of components to detect errors

within interfaces. In AR applications, several components are interconnected to process specific tasks. For example, within our virtual pet application, the 3D model is equipped with an animator component that coordinates animation changes. A gesture input triggering a movement of the virtual pet requires an interface between the gesture processor and the animation component that implements the transformation of the virtual object. This complexity generally occurs in domains where dynamic 3D objects are created, such as in game development. To efficiently test the cooperation between such individual components, conventional integration tests may need to be extended, especially when AR applications comprise complex virtual objects with interwined dependencies.

System tests. System testing verifies the adequacy of an overall system with regard to specific requirements, based on requirement documents, specifications, or user manuals. In this context, system tests often include quality attributes, such as the system performance.

In AR research, Robert et al. [97] conducted a preliminary system test for an AR guiding tool, in which gestures from a remote expert are shown on a display in real time. During robustness and performance testing, system latencies as well as video and audio transmission are verified. Furthermore, Nebeling et al. [88] designed an analysis tool enabling AR performance tests, whereas Sluganovic et al. [104] evaluated the performance of a pairing protocol for shared AR experiences by emphasized the exchange of spatial data as an influencing factor.

Regarding system tests within our AR Pong project, the synchronization of real world data revealed different response times per session. Furthermore, we found the devices to be able to share the same virtual objects, but these objects sometimes differed in their real world positions. Considering the physical world and testing the resilience of real world information was also challenging in our virtual pet application. Here the virtual pet was supposed to appear on a random surface in real space, but the registration of the physical world was unreliable and difficult to reproduce in testing. These technological limitations coincide with the observations in AR literature, where the registration of the physical world is one of the largest research topics [11, 70].

Acceptance tests. In acceptance tests, a software is evaluated from the perspective of stakeholders by reviewing the implementation against expectations and contractual agreements. These tests are usually applied early in software development in order to avoid extensive changes. In order to increase system acceptance, acceptance tests are typically conducted with several stakeholders.

In the AR context, the nature of specific AR devices leads to challenges in conducting acceptance tests. While mobile AR applications on smartphones or tablets can easily be streamed to desktop PCs to be presented to multiple persons, the visual output of HMDs is only visible to the person carrying the device. Video transmission from the user's point of view, as offered by some manufacturers, was found to be unstable, lacking in image quality, and therefore unsuitable for acceptance tests involving stakeholders. An example of the streaming functionality provided by the Microsoft HoloLens, trialed for acceptance tests within our virtual pet development, is shown in Figure 9. Here, approaches are needed that enable the efficient and stable presentation of AR applications to a large group of stakeholders.

7.2 Interaction testing and test automation

The UX of an application is a crucial factor for the application's success, essentially influenced by interaction possibilities. While testing interactive AR applications, software developers face many challenges. Corresponding activities need to consider modalities like gestures and speech, which are widely used in AR interaction concepts. Furthermore, recent AR research confirms hand-free interaction on HMD devices as a valid alternative to gesture and speech input, requiring gaze and head movement to be included in interaction testing (e.g., [103, 114]).

In software development, test automation is desired to minimize the manual testing effort. Nevertheless, for test automation purposes, user actions need to be described. Several approaches to model and formalize the description of gestures have been developed in recent years, mainly in the area of multi-touch gestures common on mobile devices. Some examples are Proton [71, 72], a textual notation, Monox [41], a graphical notation, and GestureCards [56, 57, 59], a hybrid notation mixing textual and graphical elements. These notations, however, have to be extended into 3D space to be suitable for testing AR applications. Spatial gestures can be described, for example, with GISpL [32] or GDL [48]. A gesture simulator for suitable testing, however, must also consider the fuzziness common to gesture-based interaction, which requires to analyze user behavior and define target groups. Thus, user profiles must be generated and tests be repeated several times to cover a wide range of potential gesture executions (hence the necessity for test automation) and include gesture variance specific to desired user groups [52]. Same holds true for testing speech input. While text-to-speech-engines can be used to automatically generate the necessary input, user variance must be included.

Formal descriptions of user interaction can serve as a foundation for automated interaction tests, as demonstrated by Hesenius et al. [54] for multi-touch applications. Here, artificial streams of gesture expressions defined using GeForMT [64] were integrated into a testing framework to automatically test mobile applications. This approach was also extended on a conceptual level to automate tests for multimodal applications [55]. Griebe et al. [46] also demonstrated how to test speech-based applications by injecting fake audio data into running applications. Furthermore, Griebe et al. [47] also address the automated testing of mobile sensor data in general and describe an interface for the simulation and injection of spoofed sensor data into applications on operating system level.

The aforementioned approaches have to be transferred into the development of AR applications in order to avoid laborious manual testing. In practice, software engineers are unlikely to implement

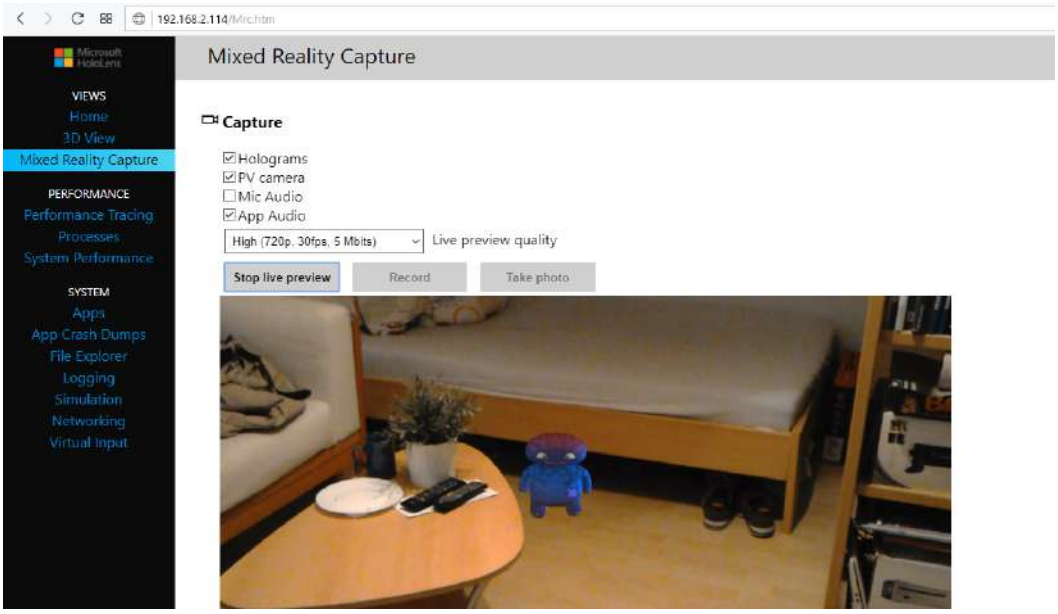

Fig. 9. Streaming of HMD content to an external device

these approaches themselves, but will need to rely on the integration of these approaches into common AR frameworks. Bernaschina et al. [9] made a first step in this direction by integrating a capture and replay framework into the evaluation of an AR outdoor application. Here, tests were conducted recording real-time data streams from multiple device sensors. Nevertheless, we assume that AR-specific characteristics like an accurate simulation of the physical environment or the consideration of further interaction modalities need to be considered in automated AR testing activities. We see an increased need for novel AR frameworks that incorporate physical elements as well as the many different user inputs and allow them to be tested for user acceptance.

7.3 User-based usability testing

Within the software engineering life cycle, usability testing is usually performed within user-based studies, where participants interact with a specific software. System usability data is measured quantitatively or qualitatively and improvements derived accordingly. Usability tests can also be performed as part of an acceptance test to fulfill system requirements [96].

In AR research, usability testing is regarded as a key activity in the evaluation of AR systems [11]. Gabbard and Swan [39] state that user-based studies are essential to derive design guidelines for the future development of AR applications. De Paiva Guimaraes and Martins [26] developed a checklist to measure the usability of AR applications, adapting common usability measurement tools from desktop applications. In recent years, AR user-based studies were carried out in many different ways. Kostaras and Xenos [74] classified various methods used in AR usability studies as *Inquiry*, *Inspection*, and *Testing* methods. Dünser and Billinghurst [29] further specify that most AR evaluations can be categorized as *Perception and Cognition*, *User Task Performance*, *Collaboration Between Users*, and *System Usability*. They additionally differentiate between several methods applied in AR evaluations, such as objective and subjective measurements or qualitative analyses. There are numerous user-based AR studies in which multiple evaluation methods were combined. For example, Billinghurst et al. [12] investigated whether conventional display-based or projection-based interfaces facilitate user collaboration, based on observations, video analysis, and the quantitative evaluation of task performance. Data collection in AR studies was also addressed by Nebeling et al. [88], presenting the *Mixed Reality Analytics Toolkit* (MRAT), allowing for data collection of objective measurements, such as the user's position or task completion times.

In AR user-based studies, the physical environment usually has an essential influence on the evaluation. As Kostaras and Xenos [74] state, the dependence of AR applications on the physical world influences the evaluation process. In a laboratory setting, this usually results in an intensive preparation of the environment in order to conduct a study. Ratcliffe et al. [94] suggest that there are AR applications that can be tested in situ and do not need the laboratory for user studies, but this only applies to suitable applications. However, within outdoor environments, it is nearly impossible to ensure constant and controlled conditions, including lighting conditions which are crucial for the registration of the physical world (see Section 6.3). This problem has been addressed in AR research, e.g. by simulating AR systems, allowing to examine different simulated physical environments under varying conditions. The simulation of physical environments becomes especially relevant when environments are safety-critical or hard to reach. Burova et al. [20] propose the simulation of these environments within VR to test AR applications. Due to the significance of the physical world in AR applications, aspects of accessibility require special attention. This is already implied in a few papers, like the consideration of physical constraints and minimizing the user input (see [2]), but should be addressed more explicitly in future user-based testing.

While testing our virtual pet application, we conducted a user-based evaluation involving five participants, each of whom individually tested the application on an HMD. The remaining participants observed the application from the user's point of view, using the streaming functionality

of the Microsoft HoloLens (see Section 7.1). We particularly examined the exploration of basic interaction concepts and the perception of specific UI elements by asking participants to perform different tasks within the AR. Participants observing the video stream were additionally asked to articulate thoughts using the *Think Aloud* method [80]. After the interaction, we conducted short interviews to identify potential for improvement. We repeated this test procedure in subsequent iterations in order to evaluate added features and identify new requirements.

In sum, we found the evaluation of our AR projects to require special organizational and time effort. During evaluation sessions, several persons had to be present to prepare the study environment and perform the evaluation process. In addition, within our virtual pet project, an initial training phase was required for each participant, in which the functionality of the device had to be introduced, due to the novelty of HMDs. Furthermore, a stable infrastructure was required to enable a live video stream from the HMD device to an external source. We observed issues regarding time delay and image quality during our evaluation sessions.

7.4 Findings

We found AR-specific challenges to be underrepresented in current testing techniques, especially regarding the consideration of the physical environment as well as novel interaction methods.

Regarding conventional testing activities, we observed many different components within our AR projects such as gesture processing, animation routines, 3D transformation programs, or spatial environment recognition to interact with each other. The consideration of the interoperability of these components and interfaces needs to be investigated in AR-specific unit and integration tests. Furthermore, we found the exchange and verification of spatial and real world data as well as data about virtual objects to be a core challenge in system tests. However, frameworks supporting the evaluation of these AR-specific information are scarce (7a).

As mentioned above, the testing of interaction possibilities is essential for quality assurance, since interaction concepts are a core aspect of the UX, especially when involving novel interaction methods. In current research, formal languages to describe gestures are applied in test automation to automatically generate and test gestures. Nevertheless, the inclusion of novel AR interactions in formal descriptions is still underrepresented (7b). Additional approaches address the simulation of sensor data to process automated test activities, supporting the rapid testing of context-sensitive applications. However, approaches for the automation of AR test activities are scarce, especially when it comes to simulating physical environments (7c).

User-based studies are broadly applied in the evaluation of AR applications, since they are essential to derive design guidelines. For many AR applications, the results of user-based studies are highly dependent on the user's physical environment. In our AR projects, we found the preparation of physical environments to be laborious, which is compounded when several physical environments are to be considered. Approaches that allow the simplified simulation of AR applications within artificial environments may be applied to enable reproducible user-based studies (7d, 7e).

During the acceptance tests within our AR projects, we observed some issues, especially regarding the presentation of results to a larger group of stakeholders. Here, streaming the AR camera image to an external device was helpful in principle, but unstable in application. Stakeholders were generally able to test AR applications individually, while the other stakeholders observed the interaction, but advances in stability and ergonomics need to be achieved (7f).

Research Agenda

7a. How can heterogeneous AR components and their interoperability be considered in unit, integration and system tests?
7b. How can novel AR interaction modalities be formally defined in a test case specification language?
7c. How can real world conditions be considered in automated tests for AR applications?
7d. How can multiple physical environments be efficiently considered in AR studies?
7e. How can environmental conditions be simulated to enable valid AR studies in reproducible environments?
7f. How can AR applications be efficiently and robustly presented to a larger group of stakeholders?

8 CONCLUSION & FUTURE WORK

We discussed software engineering activities for AR application development by highlighting current research results and combining them with our hands-on experiences from different AR projects. For each development phase, we addressed essential methods and tools and revealed research questions that still need to be investigated.

In requirements engineering, we examined brainstorming, structured workshops, and interviews as core activities. Within the UI design phase, we concentrated on sketching, prototyping, and storyboarding. In the implementation phase for AR applications, we mainly focused on the visualization of AR system components, the modeling of system behavior using diagrams, and supporting tools and frameworks. The localization and registration of the physical world as well as debugging were investigated as further core activities of the implementation phase. Our consideration of the testing phase further included unit tests, interaction tests, test automation, and usability testing.

Our paper proposes several open research questions per development phase, which we consider to be particularly relevant for future AR research and development. For early development phases such as requirements engineering, we especially propose future work to address the collaboration of technical AR experts, domain experts, and end users. In UI design, aspects of physical environments need to be emphasized, as in prototyping activities, where UI designers additionally need to be supported in the construction of digital prototypes. When it comes to implementation, the relevance of the physical environment needs to become a more central topic as well. Here, approaches are needed that simplify the simulation of the physical world, or that allow the implementation of certain components within the augmented reality. Finally, we see enormous research potential in interaction testing and test automation for AR applications.

REFERENCES

[1] 2017. ISO/IEC/IEEE International Standard - Systems and software engineering–Vocabulary. *ISO/IEC/IEEE 24765:2017(E)* (2017), 1–541. https://doi.org/10.1109/IEEESTD.2017.8016712

[2] 2020. UX Design Principles for Augmented Reality | Adobe XD Ideas. (2020). https://xd.adobe.com/ideas/principles/emerging-technology/ux-design-principles-for-augmented-reality/

[3] Daniel F. Abawi, Ralf Dörner, Michael Haller, and Jürgen Zauner. 2004. Efficient mixed reality application development. In *1st European Conference on Visual Media Production, CVMP 2004.* 289–294.

[4] Günter Alce, Klas Hermodsson, and Mattias Wallergård. 2013. WozARd: A Wizard of Oz Tool for Mobile AR. In *Proceedings of the 15th International Conference on Human-computer Interaction with Mobile Devices and Services (MobileHCI '13).* ACM, New York, NY, USA, 600–605. https://doi.org/10.1145/2493190.2494424

[5] Margarita Anastassova, Christine Mégard, and Jean-Marie Burkhardt. 2007. Prototype Evaluation and User-Needs Analysis in the Early Design of Emerging Technologies. In *Human-Computer Interaction. Interaction Design and Usability*, Julie A. Jacko (Ed.). Springer Berlin Heidelberg, Berlin, Heidelberg, 383–392.

[6] Narges Ashtari, Andrea Bunt, Joanna McGrenere, Michael Nebeling, and Parmit K. Chilana. 2020. Creating Augmented and Virtual Reality Applications: Current Practices, Challenges, and Opportunities. In *Proceedings of the 2020 CHI Conference on Human Factors in Computing Systems (CHI '20)*. Association for Computing Machinery, Honolulu, HI, USA, 1–13. https://doi.org/10.1145/3313831.3376722

[7] Tim Bailey and Hugh Durrant-Whyte. 2006. Simultaneous localization and mapping (SLAM): part II. *IEEE Robotics Automation Magazine* 13, 3 (Sept. 2006), 108–117. https://doi.org/10.1109/MRA.2006.1678144

[8] David Benyon. 2010. *Designing Interactive Systems: A Comprehensive Guide to HCI and Interaction Design* (2. ed.). Pearson Education Limited, Boston.

[9] Carlo Bernaschina, Roman Fedorov, Darian Frajberg, and Piero Fraternali. 2017. A Framework for Regression Testing of Outdoor Mobile Applications. In *Proceedings of the 4th International Conference on Mobile Software Engineering and Systems (MOBILESoft '17)*. IEEE Press, Piscataway, NJ, USA, 179–181. https://doi.org/10.1109/MOBILESoft.2017.13

[10] Allen Bierbaum, Patrick Hartling, and Carolina Cruz-Neira. 2003. Automated Testing of Virtual Reality Application Interfaces. In *Proceedings of the Workshop on Virtual Environments 2003 (EGVE '03)*. Association for Computing Machinery, New York, NY, USA, 107–114. https://doi.org/10.1145/769953.769966

[11] Mark Billinghurst, Adrian Clark, and Gun Lee. 2015. A Survey of Augmented Reality. *Foundations and Trends in Human-Computer Interaction* 8, 2-3 (2015), 73–272.

[12] Mark Billinghurst, Hirokazu Kato, Kiyoshi Kiyokawa, Daniel Belcher, and Ivan Poupyrev. 2002. Experiments with Face-To-Face Collaborative AR Interfaces. *Virtual Reality* 6, 3 (01 Oct 2002), 107–121. https://doi.org/10.1007/s100550200012

[13] Barry W. Boehm. 1979. Guidelines for Verifying and Validating Software Requirements and Design Specifications. In *Euro IFIP 79*, P. A. Samet (Ed.). North Holland, 711–719.

[14] Matthias Book, Volker Gruhn, and Rüdiger Striemer. 2016. *Tamed Agility: Pragmatic Contracting and Collaboration in Agile Software Projects* (1st ed.). Springer, New York, NY.

[15] Ingo Börsting and Volker Gruhn. 2018. Towards Rapid Digital Prototyping for Augmented Reality Applications. In *Proceedings of the 4th International Workshop on Rapid Continuous Software Engineering (RCoSE '18)*. ACM, New York, NY, USA, 12–15. https://doi.org/10.1145/3194760.3194762

[16] Ingo Börsting and Volker Gruhn. 2020. Towards Efficient Interdisciplinary Authoring of Industrial Augmented Reality Applications. In *Conference Companion of the 4th International Conference on Art, Science, and Engineering of Programming (<programming> '20)*. Association for Computing Machinery, New York, NY, USA, 65–68. https://doi.org/10.1145/3397537.3398474

[17] Pierre Bourque, Richard E. Fairley, and IEEE Computer Society. 2014. *Guide to the Software Engineering Body of Knowledge (SWEBOK(R)): Version 3.0* (3rd ed.). IEEE Computer Society Press, Washington, DC, USA.

[18] Wolfgang Broll, Irma Lindt, Jan Ohlenburg, Iris Herbst, Michael Wittkamper, and Thomas Novotny. 2005. An infrastructure for realizing custom-tailored augmented reality user interfaces. *IEEE Transactions on Visualization and Computer Graphics* 11, 6 (Nov. 2005), 722–733. https://doi.org/10.1109/TVCG.2005.90

[19] Hagen Buchholz, Constantin Brosda, and Richard Wetzel. 2010. Science Center To Go – A Mixed Reality Learning Environment of Miniature Exhibits. *Proceedings of the "Learning with ATLAS@ CERN" Workshops Inspiring Science Learning EPINOIA*, 85–96.

[20] Alisa Burova, John Mäkelä, Jaakko Hakulinen, Tuuli Keskinen, Hanna Heinonen, Sanni Siltanen, and Markku Turunen. 2020. Utilizing VR and Gaze Tracking to Develop AR Solutions for Industrial Maintenance. In *Proceedings of the 2020 CHI Conference on Human Factors in Computing Systems (CHI '20)*. Association for Computing Machinery, New York, NY, USA, 1–13. https://doi.org/10.1145/3313831.3376405

[21] Bill Buxton. 2007. *Sketching User Experiences: Getting the Design Right and the Right Design*. Morgan Kaufmann.

[22] Tenzin Choedon and Young-Chan Lee. 2018. Classification and Evaluation of Service Requirements in Mobile Tourism Application Using Kano Model and AHP. *The Journal of Information Systems* 27, 1 (2018), 43–65. https://doi.org/10.5859/KAIS.2018.27.1.43

[23] Dai Clegg and Richard Barker. 1994. *Case Method Fast-Track: A Rad Approach*. Addison-Wesley Longman Publishing Co., Inc., USA.

[24] Matthew Conway, Steve Audia, Tommy Burnette, Dennis Cosgrove, and Kevin Christiansen. 2000. Alice: Lessons Learned from Building a 3D System for Novices. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems (CHI '00)*. ACM, New York, NY, USA, 486–493. https://doi.org/10.1145/332040.332481

[25] A. Damala and N. Stojanovic. 2012. Tailoring the Adaptive Augmented Reality (A2R) museum visit: Identifying Cultural Heritage professionals' motivations and needs. In *2012 IEEE International Symposium on Mixed and Augmented Reality - Arts, Media, and Humanities (ISMAR-AMH)*. 71–80.

[26] Marcelo De Paiva Guimaraes and Valeria Farinazzo Martins. 2014. A Checklist to Evaluate Augmented Reality Applications. In *2014 XVI Symposium on Virtual and Augmented Reality*. 45–52. https://doi.org/10.1109/SVR.2014.17

[27] Marco de Sá and Elizabeth Churchill. 2012. Mobile Augmented Reality: Exploring Design and Prototyping Techniques. In *Proceedings of the 14th International Conference on Human-computer Interaction with Mobile Devices and Services*

(MobileHCI '12). ACM, New York, NY, USA, 221–230. https://doi.org/10.1145/2371574.2371608

[28] John J. Dudley, Jason T. Jacques, and Per Ola Kristensson. 2021. Crowdsourcing Design Guidance for Contextual Adaptation of Text Content in Augmented Reality. In *Proceedings of the 2021 CHI Conference on Human Factors in Computing Systems*. Number 731. Association for Computing Machinery, New York, NY, USA, 1–14. https://doi.org/10.1145/3411764.3445493

[29] Andreas Dünser and Mark Billinghurst. 2011. *Evaluating Augmented Reality Systems*. 289–307. https://doi.org/10.1007/978-1-4614-0064-6_13

[30] Andreas Dünser, Raphael Grasset, Hartmut Seichter, and Mark Billinghurst. 2007. Applying HCI principles to AR systems design. (2007). https://ir.canterbury.ac.nz/handle/10092/2340

[31] Paloma Díaz and Ignacio Aedo. 2020. Combining Software Engineering and Design Thinking Practices in the Ideation Process of Augmented Digital Experiences. *Interacting with Computers* (10 2020). https://doi.org/10.1093/iwc/iwaa020

[32] Florian Echtler and Andreas Butz. 2012. GISpL: Gestures Made Easy. In *Proceedings of the Sixth International Conference on Tangible, Embedded and Embodied Interaction (TEI '12)*. ACM, New York, NY, USA, 233–240. http://dl.acm.org/citation.cfm?id=2148181

[33] Ralf Fahney, Andrea Herrmann, and Rüdiger Weissbach. 2007. Eine neue Dimension, um zwischen Requirements Engineering und Projektmanagement zu unterscheiden (Bericht des Arbeitskreises „RE und PM"). *Softwaretechnik-Trends* 27, 1 (2007), 2.

[34] Valéria Farinazzo Martins, Paulo N. M. Sampaio, Fernanda da S. Mendes, André Santos Lima, and Marcelo de Paiva Guimarães. 2016. Usability and Functionality Assessment of an Oculus Rift in Immersive and Interactive Systems Using Voice Commands. In *Virtual, Augmented and Mixed Reality (Lecture Notes in Computer Science)*, Stephanie Lackey and Randall Shumaker (Eds.). Springer International Publishing, 222–232.

[35] Vittorio Ferrari, Tinne Tuytelaars, and Luc Van Gool. 2001. Markerless augmented reality with a real-time affine region tracker. In *Proceedings IEEE and ACM International Symposium on Augmented Reality*. 87–96. https://doi.org/10.1109/ISAR.2001.970518

[36] Jonathan S. Fish and Jonathan Stephen Scrivener. 1990. Amplifying the Mind's Eye: Sketching and Visual Cognition. *Leonardo* 23 (1990), 117 – 126.

[37] Gabriel Freitas, Márcio Sarroglia Pinho, Milene Selbach Silveira, and Frank Maurer. 2020. A Systematic Review of Rapid Prototyping Tools for Augmented Reality. *2020 22nd Symposium on Virtual and Augmented Reality (SVR)* (2020), 199–209.

[38] Henry Fuchs, Mark A. Livingston, Ramesh Raskar, D'nardo Colucci, Kurtis Keller, Andrei State, Jessica R. Crawford, Paul Rademacher, Samuel H. Drake, and Anthony A. Meyer. 1998. Augmented reality visualization for laparoscopic surgery. In *Medical Image Computing and Computer-Assisted Intervention — MICCAI'98 (Lecture Notes in Computer Science)*, William M. Wells, Alan Colchester, and Scott Delp (Eds.). Springer Berlin Heidelberg, 934–943.

[39] Joseph L. Gabbard and J. Edward Swan. 2008. Usability Engineering for Augmented Reality: Employing User-Based Studies to Inform Design. *IEEE Transactions on Visualization and Computer Graphics* 14, 3 (May 2008), 513–525. https://doi.org/10.1109/TVCG.2008.24

[40] Maribeth Gandy and Blair MacIntyre. 2014. Designer's Augmented Reality Toolkit, Ten Years Later: Implications for New Media Authoring Tools. In *Proceedings of the 27th Annual ACM Symposium on User Interface Software and Technology (UIST '14)*. ACM, New York, NY, USA, 627–636. https://doi.org/10.1145/2642918.2647369

[41] Roman Ganhör and Wolfgang Spreicer. 2014. Monox: Extensible Gesture Notation for Mobile Devices. In *Proceedings of the 16th international conference on Human-computer interaction with mobile devices & services (MobileHCI '14)*. ACM, New York, NY, USA, 203–212. https://doi.org/10.1145/2628363.2628394

[42] Qing Hong Gao, Tao Ruan Wan, Wen Tang, and Long Chen. 2017. A Stable and Accurate Marker-Less Augmented Reality Registration Method. In *2017 International Conference on Cyberworlds (CW)*. 41–47. https://doi.org/10.1109/CW.2017.44

[43] Danilo Gasques, Janet G. Johnson, Tommy Sharkey, and Nadir Weibel. 2019. What You Sketch Is What You Get: Quick and Easy Augmented Reality Prototyping with PintAR. In *Extended Abstracts of the 2019 CHI Conference on Human Factors in Computing Systems (CHI EA '19)*. Association for Computing Machinery, New York, NY, USA, 1–6. https://doi.org/10.1145/3290607.3312847

[44] Pierre Georgel, Pierre Schroeder, Selim Benhimane, Stefan Hinterstoisser, Mirko Appel, and Nassir Navab. 2007. An Industrial Augmented Reality Solution For Discrepancy Check. In *Proceedings of the 2007 6th IEEE and ACM International Symposium on Mixed and Augmented Reality (ISMAR '07)*. IEEE Computer Society, Washington, DC, USA, 1–4. https://doi.org/10.1109/ISMAR.2007.4538834

[45] Saul Greenberg, Sheelagh Carpendale, Nicolai Marquardt, and Bill Buxton. 2011. *Sketching User Experiences: The Workbook* (1st ed.). Morgan Kaufmann Publishers Inc., San Francisco, CA, USA.

[46] Tobias Griebe, Marc Hesenius, Marc Gesthüsen, and Volker Gruhn. 2016. Test Automation for Speech-Based Applications. In *New Trends in Software Methodologies, Tools and Techniques: Proceedings of the Fifteenth SoMeT_16.*

IOS Press, 85–100. https://doi.org/10.3233/978-1-61499-674-3-85

[47] Tobias Griebe, Marc Hesenius, and Volker Gruhn. 2015. Towards Automated UI-Tests for Sensor-Based Mobile Applications. In *Intelligent Software Methodologies, Tools and Techniques: 14th International Conference, SoMet 2015, Naples, Italy, September 15-17, 2015. Proceedings*, Hamido Fujita and Guido Guizzi (Eds.). Communications in Computer and Information Science, Vol. 532. Springer International Publishing, 3–17. https://doi.org/10.1007/978-3-319-22689-7_1

[48] Tomasz Hachaj and Marek R. Ogiela. 2014. Rule-Based Approach to Recognizing Human Body Poses and Gestures in Real Time. *Multimedia Systems* 20, 1 (2014), 81–99. https://doi.org/10.1007/s00530-013-0332-2

[49] Nate Hagbi, Raphaet Grasset, Oriel Bergig, Mark Billinghurst, and Jihad El-Sana. 2010. In-Place Sketching for Content Authoring in Augmented Reality Games. In *Proceedings of the 2010 IEEE Virtual Reality Conference (VR '10)*. IEEE Computer Society, USA, 91–94. https://doi.org/10.1109/VR.2010.5444806

[50] David Harel. 1987. Statecharts: A Visual Formalism for Complex Systems. *Sci. Comput. Program.* 8, 3 (6 1987), 231–274.

[51] John Hart. 1998. *The Art of the Storyboard: Storyboarding for Film, TV, and Animation.* Focal Press.

[52] Marc Hesenius, Matthias Book, and Volker Gruhn. 2019. Test Automation for Gesture-Based Interfaces. In *HCI Engineering 2019 – Methods and Tools for Advanced Interactive Systems and Integration of Multiple Stakeholder Viewpoints.* CEUR Workshop Proceedings. http://ceur-ws.org/Vol-2503/paper1_5.pdf

[53] Marc Hesenius, Ingo Börsting, Ole Meyer, and Volker Gruhn. 2018. Don't Panic!: Guiding Pedestrians in Autonomous Traffic with Augmented Reality. In *Proceedings of the 20th International Conference on Human-Computer Interaction with Mobile Devices and Services Adjunct (MobileHCI '18)*. ACM, New York, NY, USA, 261–268. https://doi.org/10.1145/3236112.3236148

[54] Marc Hesenius, Tobias Griebe, Stefan Gries, and Volker Gruhn. 2014. Automating UI Tests for Mobile Applications with Formal Gesture Descriptions. In *Proceedings of the 16th International Conference on Human-computer Interaction with Mobile Devices and Services (MobileHCI '14)*. ACM, New York, NY, USA, 213–222. https://doi.org/10.1145/2628363.2628391

[55] Marc Hesenius, Tobias Griebe, and Volker Gruhn. 2014. Towards a Behavior-oriented Specification and Testing Language for Multimodal Applications. In *Proceedings of the 2014 ACM SIGCHI Symposium on Engineering Interactive Computing Systems (EICS '14)*. ACM, New York, NY, USA, 117–122. https://doi.org/10.1145/2607023.2610278

[56] Marc Hesenius and Volker Gruhn. 2016. Introducing GestureCards: A Prototyping Gesture Notation. In *Proceedings of the 9th Nordic Conference on Human-Computer Interaction (NordiCHI '16)*. ACM, New York, NY, USA, Article 111, 6 pages. https://doi.org/10.1145/2971485.2996746

[57] Marc Hesenius and Volker Gruhn. 2019. GestureCards – A Hybrid Gesture Notation. *Proceedings of the ACM on Human-Computer Interaction* 3, EICS, Article 22 (2019), 35 pages. https://doi.org/10.1145/3331164

[58] Marc Hesenius, Markus Kleffmann, and Volker Gruhn. 2016. Sketching Gesture-Based Applications in a Collaborative Working Environment with Wall-Sized Displays. In *2016 International Conference on Collaboration Technologies and Systems (CTS)*. 327–336. https://doi.org/10.1109/CTS.2016.0066

[59] Marc Hesenius, Simon Sternal, and Volker Gruhn. 2017. A Multi-Touch-Recognizer for GestureCards. In *Proceedings of the ACM SIGCHI Symposium on Engineering Interactive Computing Systems (EICS '17)*. ACM, New York, NY, USA, 75–80. https://doi.org/10.1145/3102113.3102132

[60] Ann M. Hickey, Alan M. Davis, and Denali Kaiser. 2003. Requirements Elicitation Techniques: Analyzing the Gap between Technology Availability and Technology Use. *Comparative Technology Transfer and Society* 1, 3 (Dec. 2003), 279–302. https://doi.org/10.1353/ctt.2003.0026

[61] Ali Hosseini-Khayat, Teddy Seyed, Chris Burns, and Frank Maurer. 2011. Low-fidelity Prototyping of Gesture-based Applications. In *Proceedings of the 3rd ACM SIGCHI Symposium on Engineering Interactive Computing Systems (EICS '11)*. ACM, New York, NY, USA, 289–294. https://doi.org/10.1145/1996461.1996538

[62] Florian Jasche, Sven Hoffmann, Thomas Ludwig, and Volker Wulf. 2021. Comparison of Different Types of Augmented Reality Visualizations for Instructions. In *Proceedings of the 2021 CHI Conference on Human Factors in Computing Systems*. Association for Computing Machinery, New York, NY, USA, Article 131, 13 pages. https://doi.org/10.1145/3411764.3445724

[63] Jason Jerald. 2016. *The VR Book: Human-Centered Design for Virtual Reality.* ACM and Morgan & Claypool, New York, NY, USA.

[64] Dietrich Kammer, Dana Henkens, C. Henzen, and Rainer Groh. 2015. Gesture Formalization for Multitouch. *Software: Practice and Experience* 45, 4 (2015), 527–548. https://doi.org/10.1002/spe.2247

[65] Seokbin Kang, Ekta Shokeen, Virginia L. Byrne, Leyla Norooz, Elizabeth Bonsignore, Caro Williams-Pierce, and Jon E. Froehlich. 2020. ARMath: Augmenting Everyday Life with Math Learning. In *Proceedings of the 2020 CHI Conference on Human Factors in Computing Systems (CHI '20)*. Association for Computing Machinery, New York, NY, USA, 1–15. https://doi.org/10.1145/3313831.3376252

[66] John A. Karasinski, Richard Joyce, Colleen Carroll, Jack Gale, and Steven Hillenius. 2017. An Augmented Reality/Internet of Things Prototype for Just-in-time Astronaut Training. In *Virtual, Augmented and Mixed Reality (Lecture Notes in Computer Science)*. Springer, Cham, 248–260. https://doi.org/10.1007/978-3-319-57987-0_20

[67] Hirokazu Kato, Keihachiro Tachibana, Mark Billinghurst, and Michael Grafe. 2003. A registration method based on texture tracking using ARToolKit. In *2003 IEEE International Augmented Reality Toolkit Workshop*. 77–85. https://doi.org/10.1109/ART.2003.1320435

[68] Gini Keating, Daniel Guest, Anne Konertz, Niccolo Padovani, and Andrea Villa. 2011. Designing the AR Experience: Tools and Tips for Mobile Augmented Reality UX Design. In *Design, User Experience, and Usability. Theory, Methods, Tools and Practice (Lecture Notes in Computer Science)*. Springer, Berlin, Heidelberg, 135–141. https://doi.org/10.1007/978-3-642-21708-1_16

[69] G. Jounghyun Kim, Kyo Chul Kang, Hyejung Kim, and Jiyoung Lee. 1998. Software Engineering of Virtual Worlds. In *Proceedings of the ACM Symposium on Virtual Reality Software and Technology (VRST '98)*. ACM, New York, NY, USA, 131–138. https://doi.org/10.1145/293701.293718

[70] Kangsoo Kim, Mark Billinghurst, Gerd Bruder, Henry Been-Lirn Duh, and Gregory F. Welch. 2018. Revisiting Trends in Augmented Reality Research: A Review of the 2nd Decade of ISMAR (2008–2017). *IEEE Transactions on Visualization and Computer Graphics* 24, 11 (2018), 2947–2962.

[71] Kenrick Kin, Björn Hartmann, Tony DeRose, and Maneesh Agrawala. 2012. Proton++: A Customizable Declarative Multitouch Framework. In *Proceedings of the 25th Annual ACM Symposium on User Interface Software and Technology* (2012). ACM, 477–486. https://doi.org/10.1145/2380116.2380176

[72] Kenrick Kin, Björn Hartmann, Tony DeRose, and Maneesh Agrawala. 2012. Proton: Multitouch Gestures as Regular Expressions. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems*. Association for Computing Machinery, New York, NY, USA, 2885–2894. https://doi.org/10.1145/2207676.2208694

[73] Sang Min Ko, Won Suk Chang, and Yong Gu Ji. 2013. Usability Principles for Augmented Reality Applications in a Smartphone Environment. *International Journal of Human–Computer Interaction* 29, 8 (Aug. 2013), 501–515. https://doi.org/10.1080/10447318.2012.722466

[74] Nektarios Kostaras and Michalis Xenos. 2009. Assessing the Usability of Augmented Reality Systems. In *Proceedings of The 13th Panhellenic Conference on Informatics (PCI 2009)*. Corfu, Greece, 197-201.

[75] Veronika Krauß, Alexander Boden, Leif Oppermann, and René Reiners. 2021. Current Practices, Challenges, and Design Implications for Collaborative AR/VR Application Development. In *Proceedings of the 2021 CHI Conference on Human Factors in Computing Systems (CHI '21)*. Association for Computing Machinery, New York, NY, USA, 1–15. https://doi.org/10.1145/3411764.3445335

[76] Tobias Langlotz, Stefan Mooslechner, Stefanie Zollmann, Claus Degendorfer, Gerhard Reitmayr, and Dieter Schmalstieg. 2011. Sketching up the world: In situ authoring for mobile Augmented Reality. *Personal and Ubiquitous Computing* 16 (08 2011), 1–8. https://doi.org/10.1007/s00779-011-0430-0

[77] Felix Lauber, Claudius Böttcher, and Andreas Butz. 2014. PapAR: Paper Prototyping for Augmented Reality. In *Adjunct Proceedings of the 6th International Conference on Automotive User Interfaces and Interactive Vehicular Applications (AutomotiveUI '14)*. ACM, New York, NY, USA, 1–6. https://doi.org/10.1145/2667239.2667271

[78] Gun A. Lee, Gerard Jounghyun Kim, and Chan-Mo Park. 2002. Modeling virtual object behavior within virtual environment. In *Proceedings of the ACM symposium on Virtual reality software and technology - VRST '02*. ACM Press, Hong Kong, China, 41. https://doi.org/10.1145/585740.585748

[79] Germán Leiva, Cuong Nguyen, Rubaiat Habib Kazi, and Paul Asente. 2020. Pronto: Rapid Augmented Reality Video Prototyping Using Sketches and Enaction. In *Proceedings of the 2020 CHI Conference on Human Factors in Computing Systems (CHI '20)*. Association for Computing Machinery, New York, NY, USA, 1–13. https://doi.org/10.1145/3313831.3376160

[80] Clayton Lewis. 1982. *Using the "thinking Aloud" Method in Cognitive Interface Design*. IBM T.J. Watson Research Center, Yorktown, NY, USA.

[81] Linchuan Liu and Peter Khooshabeh. 2003. Paper or Interactive?: A Study of Prototyping Techniques for Ubiquitous Computing Environments. In *CHI '03 Extended Abstracts on Human Factors in Computing Systems (CHI EA '03)*. ACM, New York, NY, USA, 1030–1031. https://doi.org/10.1145/765891.766132

[82] Dan Lockton and Carine Lallemand. 2020. Meeting Designers Where They Are: Using Industry Events as a Research Venue for HCI and Design Methods Development. In *Proceedings of the 2020 CHI Conference on Human Factors in Computing Systems (CHI '20)*. Association for Computing Machinery, Honolulu, HI, USA, 1–13. https://doi.org/10.1145/3313831.3376206

[83] Blair MacIntyre, Maribeth Gandy, Steven Dow, and Jay David Bolter. 2004. DART: A Toolkit for Rapid Design Exploration of Augmented Reality Experiences. In *Proceedings of the 17th Annual ACM Symposium on User Interface Software and Technology (UIST '04)*. ACM, New York, NY, USA, 197–206. https://doi.org/10.1145/1029632.1029669

[84] Robert C. Martin. 2007. Professionalism and Test-Driven Development. *IEEE Software* 24, 3 (May 2007), 32–36. https://doi.org/10.1109/MS.2007.85

[85] Héctor Martínez, Danai Skournetou, Jenni Hyppölä, Seppo Laukkanen, and Antti Heikkilä. 2014. Drivers and Bottlenecks in the Adoption of Augmented Reality Applications. *Journal of Multimedia Theory and Applications* 2 (March 2014), 27–44. https://doi.org/10.11159/jmta.2014.004

[86] Michael McCurdy, Christopher Connors, Guy Pyrzak, Bob Kanefsky, and Alonso Vera. 2006. Breaking the Fidelity Barrier: An Examination of Our Current Characterization of Prototypes and an Example of a Mixed-fidelity Success. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems (CHI '06)*. ACM, New York, NY, USA, 1233–1242. https://doi.org/10.1145/1124772.1124959

[87] Leon Müller, Ken Pfeuffer, Jan Gugenheimer, Bastian Pfleging, Sarah Prange, and Florian Alt. 2021. SpatialProto: Exploring Real-World Motion Captures for Rapid Prototyping of Interactive Mixed Reality. In *Proceedings of the 2021 CHI Conference on Human Factors in Computing Systems*. Number 363. Association for Computing Machinery, New York, NY, USA, 1–13. https://doi.org/10.1145/3411764.3445560

[88] Michael Nebeling, Maximilian Speicher, Xizi Wang, Shwetha Rajaram, Brian D. Hall, Zijian Xie, Alexander R. E. Raistrick, Michelle Aebersold, Edward G. Happ, Jiayin Wang, Yanan Sun, Lotus Zhang, Leah E. Ramsier, and Rhea Kulkarni. 2020. MRAT: The Mixed Reality Analytics Toolkit. In *Proceedings of the 2020 CHI Conference on Human Factors in Computing Systems (CHI '20)*. Association for Computing Machinery, New York, NY, USA, 1–12. https://doi.org/10.1145/3313831.3376330

[89] Leticia Neira-Tovar and Ivan Castilla Rodriguez. 2017. A Virtual Reality Tool Applied to Improve the Effects on Chronic Diseases - Case: Emotional Effects on T2DM. In *Virtual, Augmented and Mixed Reality*, Stephanie Lackey and Jessie Chen (Eds.). Springer International Publishing, Cham, 417–425. https://doi.org/10.1007/978-3-319-57987-0_34

[90] Thomas Olsson and Markus Salo. 2012. Narratives of Satisfying and Unsatisfying Experiences of Current Mobile Augmented Reality Applications. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems (CHI '12)*. ACM, New York, NY, USA, 2779–2788. https://doi.org/10.1145/2207676.2208677

[91] Alex F. Osborn. 1953. *Applied imagination principles and procedures of creative thinking*. New York Scribner.

[92] Wayne Piekarski and Bruce H. Thomas. 2003. An object-oriented software architecture for 3D mixed reality applications. In *The Second IEEE and ACM International Symposium on Mixed and Augmented Reality, 2003. Proceedings.* 247–256. https://doi.org/10.1109/ISMAR.2003.1240708

[93] Thammathip Piumsomboon, Adrian Clark, Mark Billinghurst, and Andy Cockburn. 2013. User-Defined Gestures for Augmented Reality. In *CHI '13 Extended Abstracts on Human Factors in Computing Systems (CHI EA '13)*. Association for Computing Machinery, New York, NY, USA, 955–960. https://doi.org/10.1145/2468356.2468527

[94] Jack Ratcliffe, Francesco Soave, Nick Bryan-Kinns, Laurissa Tokarchuk, and Ildar Farkhatdinov. 2021. *Extended Reality (XR) Remote Research: A Survey of Drawbacks and Opportunities*. Association for Computing Machinery, New York, NY, USA. https://doi.org/10.1145/3411764.3445170

[95] Joseph Redmon, Santosh Kumar Divvala, Ross B. Girshick, and Ali Farhadi. 2015. You Only Look Once: Unified, Real-Time Object Detection. *CoRR* abs/1506.02640 (2015). http://arxiv.org/abs/1506.02640

[96] Michael Richter and Markus Flückiger. 2014. *User-Centred Engineering: Creating Products for Humans*. Springer Publishing Company, Incorporated.

[97] Kostia Robert, Dingyun Zhu, Weidong Huang, Leila Alem, and Tom Gedeon. 2013. MobileHelper: Remote Guiding Using Smart Mobile Devices, Hand Gestures and Augmented Reality. In *SIGGRAPH Asia 2013 Symposium on Mobile Graphics and Interactive Applications (SA '13)*. Association for Computing Machinery, New York, NY, USA, Article Article 39, 5 pages. https://doi.org/10.1145/2543651.2543664

[98] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L. Chen. 2018. MobileNetV2: Inverted Residuals and Linear Bottlenecks. In *2018 IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 4510–4520. https://doi.org/10.1109/CVPR.2018.00474

[99] Carlos Santos, Brunelli Miranda, Tiago Araujo, Nikolas Carneiro, Anderson Marques, Marcelle Mota, Jefferson Morais, and Bianchi Meiguins. 2016. Guidelines for Graphical User Interface Design in Mobile Augmented Reality Applications. In *Virtual, Augmented and Mixed Reality (Lecture Notes in Computer Science)*. Springer, Cham, 71–80. https://doi.org/10.1007/978-3-319-39907-2_7

[100] Dieter Schmalstieg and Tobias Höllerer. 2016. *Augmented Reality: Principles and Practice*. Addison-Wesley.

[101] Hartmut Seichter, Julian Looser, and Mark Billinghurst. 2008. ComposAR: An intuitive tool for authoring AR applications. In *2008 7th IEEE/ACM International Symposium on Mixed and Augmented Reality*. IEEE, Cambridge, UK, 177–178. https://doi.org/10.1109/ISMAR.2008.4637354

[102] Frank M. Shipman and Catherine C. Marshall. 1999. Formality Considered Harmful: Experiences, Emerging Themes, and Directions on the Use of Formal Representations in Interactive Systems. *Computer Supported Cooperative Work (CSCW)* 8, 4 (01 Dec 1999), 333–352. https://doi.org/10.1023/A:1008716330212

[103] Ludwig Sidenmark, Dominic Potts, Bill Bapisch, and Hans Gellersen. 2021. *Radi-Eye: Hands-Free Radial Interfaces for 3D Interaction Using Gaze-Activated Head-Crossing.* Association for Computing Machinery, New York, NY, USA. https://doi.org/10.1145/3411764.3445697

[104] Ivo Sluganovic, Matej Serbec, Ante Derek, and Ivan Martinovic. 2017. HoloPair: Securing Shared Augmented Reality Using Microsoft HoloLens. In *Proceedings of the 33rd Annual Computer Security Applications Conference (ACSAC 2017).* Association for Computing Machinery, New York, NY, USA, 250–261. https://doi.org/10.1145/3134600.3134625

[105] Eleanor Smith, Gordon Semple, Dorothy Evans, Kenneth McRae, and Paul Blackwell. 2020. Augmented Instructions: Analysis of Performance and Efficiency of Assembly Tasks. In *Virtual, Augmented and Mixed Reality. Industrial and Everyday Life Applications*, Jessie Y. C. Chen and Gino Fragomeni (Eds.). Springer International Publishing, Cham, 166–177.

[106] Ian Sommerville. 2010. *Software Engineering* (9th ed.). Addison-Wesley Publishing Company, USA.

[107] Kihoon Son, Hwiwon Chun, Sojin Park, and Kyung Hoon Hyun. 2020. C-Space: An Interactive Prototyping Platform for Collaborative Spatial Design Exploration. In *Proceedings of the 2020 CHI Conference on Human Factors in Computing Systems (CHI '20).* Association for Computing Machinery, New York, NY, USA, 1–13. https://doi.org/10.1145/3313831.3376452

[108] Andreas Spillner, Tilo Linz, and Hans Schaefer. 2011. *Software Testing Foundations: A Study Guide for the Certified Tester Exam* (3rd ed.). Rocky Nook.

[109] Gabriel Takacs, Vijay Chandrasekhar, Natasha Gelfand, Yingen Xiong, Wei-Chao Chen, Thanos Bismpigiannis, Radek Grzeszczuk, Kari Pulli, and Bernd Girod. 2008. Outdoors Augmented Reality on Mobile Phone Using Loxel-based Visual Feature Organization. In *Proceedings of the 1st ACM International Conference on Multimedia Information Retrieval (MIR '08).* ACM, New York, NY, USA, 427–434. https://doi.org/10.1145/1460096.1460165

[110] Khai N. Truong, Gillian R. Hayes, and Gregory D. Abowd. 2006. Storyboarding: An Empirical Determination of Best Practices and Effective Guidelines. In *Proceedings of the 6th Conference on Designing Interactive Systems (DIS '06).* Association for Computing Machinery, New York, NY, USA, 12–21. https://doi.org/10.1145/1142405.1142410

[111] Markus Unterauer. 2014. *Workshops im Requirements Engineering: Methoden, Checklisten und Best Practices für die Ermittlung von Anforderungen.* Dpunkt.Verlag GmbH.

[112] Guy Vollmer. 2017. *Mobile App Engineering: Eine systematische Einführung – von den Requirements zum Go Live.* dpunkt.verlag.

[113] Zeyu Wang, Cuong Nguyen, Paul Asente, and Julie Dorsey. 2021. DistanciAR: Authoring Site-Specific Augmented Reality Experiences for Remote Environments. In *Proceedings of the 2021 CHI Conference on Human Factors in Computing Systems.* Number 411. Association for Computing Machinery, New York, NY, USA, 1–12. https://doi.org/10.1145/3411764.3445552

[114] Yukang Yan, Yingtian Shi, Chun Yu, and Yuanchun Shi. 2020. HeadCross: Exploring Head-Based Crossing Selection on Head-Mounted Displays. *Proc. ACM Interact. Mob. Wearable Ubiquitous Technol.* 4, 1, Article 35 (March 2020), 22 pages. https://doi.org/10.1145/3380983

[115] Qian Zhou, Sarah Sykes, Sidney Fels, and Kenrick Kin. 2020. Gripmarks: Using Hand Grips to Transform In-Hand Objects into Mixed Reality Input. In *Proceedings of the 2020 CHI Conference on Human Factors in Computing Systems (CHI '20).* Association for Computing Machinery, New York, NY, USA, 1–11. https://doi.org/10.1145/3313831.3376313

Received April 2021; revised July 2021; accepted September 2021